



BRNO UNIVERSITY OF TECHNOLOGY

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF INFORMATION TECHNOLOGY

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

DEPARTMENT OF INTELLIGENT SYSTEMS

ÚSTAV INTELIGENTNÍCH SYSTÉMŮ

DETECTING AI-GENERATED FAKE REVIEWS IN ONLINE PLATFORMS

DETEKCE FALEŠNÝCH RECENZÍ GENEROVANÝCH AI NA ONLINE PLATFORMÁCH

BACHELOR'S THESIS

BAKALÁŘSKÁ PRÁCE

AUTHOR

AUTOR PRÁCE

IGOR LACKO

SUPERVISOR

VEDOUCÍ PRÁCE

Ing. ANTON FIRCI, Ph.D.

BRNO 2026

Abstract

This thesis aims to design and train a classifier for detecting fake reviews generated by artificial intelligence on shopping platforms, focusing on Slovak reviews. The thesis analyzes the impact of fake reviews on the reputation and sales of products and services. A synthetic dataset is created using scraped hotel reviews from Booking.com, fine-tuning a generative model on them and generating fake hotel reviews. The implemented classifier is a stacked ensemble model utilizing stylometry, TF-IDF, and an Encoder model. The model reaches F1-Scores of 0.89 and 0.83 when tested on reviews generated by the same model as the reviews it was trained on, is able to generalize somewhat well to other models and some of the base models perform strong even on targeted manipulations.

Abstrakt

Táto práca sa zameriava na návrh a trénovanie klasifikátora na detekciu falošných recenzií, konkrétne recenzií generovaných AI. Práca skúma dopad falošných recenzií na reputáciu a predaj produktov a služieb. Využíva vlastný dataset zložený z hotelových recenzií z portálu Booking.com a syntetických recenzií generovaných jazykovým modelom, ktorý bol natrénovaný na pôvodných recenziách. Natrénovaný klasifikátor je ensemble model, ktorý využíva stylometriu, TF-IDF a enkodérový model. Model dosahuje F1-Skóre až 0.89 a 0.83 pri testovaní na recenziách od rovnakého modelu na akom bol trénovaný, je schopný sa čiastočne generalizovať aj na detekciu iných generatívnych modelov a niektoré jeho individuálne modely sú odolné aj voči cieľným manipuláciám.

Keywords

Fake reviews, Machine learning, Artificial Intelligence, Text classification, Natural language processing, Python, Large language models

Klíčová slova

Falošné recenzie, Strojové učenie, Umelá inteligencia, Klasifikácia textu, Spracovanie prirodzeného jazyka, Python, Veľké jazykové modely

Reference

LACKO, Igor. *Detecting AI-Generated Fake Reviews in Online Platforms*. Brno, 2026. Bachelor's thesis. Brno University of Technology, Faculty of Information Technology. Supervisor Ing. Anton Firc, Ph.D.

Rozšířený abstrakt

Falošné recenzie sú problém, ktorý trápi online platformy pre predaj rôznych produktov a služieb už dlhú dobu. Recenzie majú pre zákazníkov obrovskú hodnotu, keďže sú využívané ako merítko kvality daného produktu. Ich hodnota je však práve v ich autenticite, čo falošné recenzie narušajú. Motivácia za falošnými recenziami je prevažne finančná. Poskytovatelia produktov a služieb môžu falšovať pozitívne recenzie na svoje produkty, alebo negatívne recenzie na produkty konkurencie. V každom prípade to spôsobuje to, že reputácia produktu je umelo manipulovaná a nezodpovedá objektívnej realite. Trpia tým hlavne zákazníci, ale aj poskytovatelia produktov a celý systém dôvery založený na recenziách. Tento problém je amplifikovaný v posledných rokoch, v dobe umelej inteligencie a predovšetkým generatívnych modelov. Podľa niektorých štúdií je väčšina súčasného obsahu na internete umelá a pochádza od rôznych botov alebo jazykových modelov. To zahŕňa aj recenzie. Falošné recenzie síce nie sú nový problém, ale model útočníkov sa značne zmenil. V minulosti falošné recenzie buď museli písať útočníci sami, alebo za to zaplatiť niekomu inému. To je značne viac pracné ako otvorenie webového portálu s jazykovým modelom a vygenerovanie 10 recenzií naraz. Taktiež rôzne štúdie ukázali, že falošné recenzie písané ľuďmi nie sú schopné skutočne napodobniť tie autentické, keďže ich text neodpovedá skutočnému názoru útočníka. Veľké jazykové modely, ktoré sú trénované na celom internete nemusia mať túto slabinu. Táto práca sa zameriava na detekciu recenzií generovaných umelou inteligenciou v slovenskom jazyku. Takmer všetky súčasné štúdie na túto tému sa zaoberajú anglickým jazykom a žiadna sa nezaobrá slovenčinou či češtinou. Táto práca za účelom detekcie najprv vytvára vlastný dataset viac než 10000 hotelových recenzií v slovenčine z portálu Booking.com. Potom ho kombinuje so syntetickými recenziami generovanými jazykovým modelom Gemma 3, ktorý bol predtým natrénovaný na pôvodných recenziách za účelom generovania kvalitných syntetických dát. Na kombinovanom datasete syntetických a reálnych recenzií je natrénovaný klasifikátor pre detekciu AI generovaných recenzií. Navrhnutý klasifikátor je stacked ensemble klasifikátor, ktorý využíva 3 modely, z ktorých každý využíva iný typ príznakov na klasifikáciu. Prvý model sa zameriava na štatistické a štylometrické príznaky, druhý model využíva TF-IDF a tretí model je enkodérový transformer ktorý vnútorne využíva embeddingy. Na výstupoch týchto modelov je natrénovaný meta model, ktorý dáva finálny výstup. Vyskúšaných bolo viac modelov spolu s hľadaním najlepších parametrov. Práca nakoniec využíva Random Forest pre štatistické príznaky, Logistickú Regresiu na TF-IDF a SlovakBERT ako enkodérový model. Meta model je opäť Random Forest. Výsledný klasifikátor bol natrénovaný na kombinovaných reálnych a syntetických recenziách a otestovaný na viacerých testovacích sadách. Základná testovacia sada sú recenzie generované modelom Gemma 3, ktoré klasifikátor nevidel počas tréningu. Taktiež bola vytvorená testovacia sada kde syntetické recenzie vygeneroval iný model, konkrétne Slovak Mistral pre overenie klasifikátoru a jeho schopnosti generalizovať sa na inú umelú inteligenciu. Boli vytvorené aj testovacie sady ktoré testujú odolnosť klasifikátoru voči cieľným manipuláciám. Jedna z nich odstraňuje interpunkciu zo syntetických recenzií a druhá náhodne nahrádza časť slov v syntetických recenziách synonymami.

Detecting AI-Generated Fake Reviews in Online Platforms

Declaration

I hereby declare that this Bachelor's thesis was prepared as an original work by the author under the supervision of Ing. Anton Firc, Ph.D. I have listed all the literary sources, publications and other sources, which were used during the preparation of this thesis. I have used large language models (ChatGPT, Gemini, Claude) for the language revision of the thesis and for explaining unfamiliar concepts. I have used Github Copilot to assist with the implementation part of the thesis. I have used Elicit.com to find relevant sources.

.....

Igor Lacko
April 27, 2026

Acknowledgements

First and foremost, i would like to thank my supervisor, Ing. Anton Firc, Ph.D for his help and guidance during the preparation of this thesis. I would also like to thank my friends and family for supporting me throughout my entire Bachelor's studies.

Contents

1	Introduction	3
2	Fake reviews	5
2.1	The influence of fake reviews	5
2.2	Generating fake reviews using AI	7
3	Machine learning and detecting AI-generated reviews	9
3.1	Basics of Machine Learning	9
3.2	Machine Learning models	10
3.3	Model evaluation metrics	16
3.4	Commonly used features in fake review detection	17
4	The Booking.com synthetic review dataset	20
4.1	Choice of the data source	20
4.2	Collecting data	21
4.3	Dataset characteristics	21
4.4	Generating synthetic data	24
4.5	Splitting the dataset for training and testing	28
5	Architecture of the classifier	30
5.1	Model components	31
5.2	The classification framework	32
5.3	Ensemble training algorithms	32
5.4	Used models	33
6	Evaluation	35
6.1	Individual model evaluation	35
6.2	Full ensemble evaluation	41
6.3	Partial ensemble evaluation	41
6.4	Other experiments	43
6.5	Practical applications	45
7	Conclusion	46
	Bibliography	47
A	Hyperparameters of used models	52
B	Guideline to reproducing experiments	59

List of Figures

1	Spammers may promote or devalue products for market advantage. Consumers look at and decide based on them.	6
2	Input features in 2D where they aren't separable by a 2D hyperplane (a line) and in 3D, where they are separable by a 3D hyperplane (a plane) [17]. . .	11
3	High level overview of Random Forests: The predictions of the trees are transformed into one prediction via majority voting or averaging [22]. . . .	12
4	Example of a neural network. Figure heavily inspired by a figure from [44].	15
5	Distribution of dataset domains used in fake review detection from 2021 to 2023	20
6	Distribution of ratings in the dataset.	22
7	Distribution of review lengths in the dataset.	23
8	Process of creating the review dataset.	28
9	Architecture of the fake review detector.	30
10	Base models are trained on $K - 1$ folds, and generate predictions on the remaining fold. This is repeated K times until the model generates predictions on all folds.	33
11	Comparison of F1-Scores across different features and dataset variants. . . .	36
12	Precision and Recall of each model compared on each variant.	38
13	Top 10 features of the statistical model sorted by importance, for both variants.	40
14	Precision, Recall, and F1-Score of the full and partial ensemble models. . .	42
15	Feature importance of the meta model (Random Forest).	42
16	Comparison of F1-Score, Precision, and Recall on SlovakMistral reviews for the full ensemble and for the individual models.	43
17	Comparison of F1-Score, Precision, and Recall on reviews without punctuation for the full ensemble and for the individual models.	44

Chapter 1

Introduction

Fake reviews have been a problem for online platforms since their inception. Reviews are often the first thing someone reads when considering buying a product or using a service. As such, they are one of the biggest influences in it's reputation, which in turn influences it's sales. Moreover, people may not even buy a quality product if it's reputation is damaged by fake reviews, or in a opposite example, may not buy a low quality product if a product is artificially devalued. It's no wonder that people or companies often try to take advantage of this, whether it's for their advantage via faking positive reviews on their products or to the disadvantage of potential competitors via faking negative reviews on their products. The „traditional“ way of faking reviews in the past has been to manually write them, or pay for someone else to do it, which is time consuming and costly due to requiring manual human labor. However, with the rise of generative language models, such as *GPT*, and the ease of access to them, fake reviews can now be mass produced for cheap. The detection of fake reviews is a continuously developing field. This is due to multiple factors: human written fake reviews differ from AI generated ones, and even among the latter, as language models keep evolving, the methods for detecting their output have to as well. As humans have been shown not to be consistently able to correctly classify reviews as real or fake, as well as manual classification not being feasible from a time perspective, there are multiple benefits to automating this process. A common way to do this is by using machine learning (ML) methods, whether via more „traditional“ ML models such as *Naive Bayes*, *Logistic regression* or *Support vector machines (SVM)*, or more recently via utilizing language models. Commonly used features for these models are stylometric ones such as text length or writing style. However, these approaches, introduce another challenge, which is collecting relevant testing and training data. Existing datasets are few and far between, are mostly in English and barring recent research, focus mostly on manual spam instead of AI-generated reviews. This work aims to create a synthetic dataset in the Slovak language by combining existing reviews written in Slovak, using a generative model to generate the „fake half“ of the dataset, and train a classifier on the created dataset for the purpose of detecting reviews generated by language models. The classifier is an ensemble model which utilizes a pre-trained BERT-like model for the Slovak language in addition to classifiers who take in various stylometric features and other textual features, such as TF-IDF. The text of this thesis is structured as follows: [Chapter 2](#) describes the problem of fake reviews, the motivations behind them, and how can they affect consumers. [Chapter 3](#) then talks about the common features and techniques used in the domain of fake review detection up to this point, including both traditional machine learning methods and detection using transformers based language models. [Chapter 4](#) describes the dataset that

was used, how the real reviews were collected, and how it's synthetic half was generated. **Chapter 5** describes the architecture of the proposed classifier, the training algorithm and the models used. **Chapter 6** then describes the conducted experiments, discusses the results, and **Chapter 7** concludes the thesis.

Chapter 2

Fake reviews

Fake reviews pose a huge problem to both customers and sellers. They reduce the trust of customers in online platforms and thus have an inherently negative commercial impact on online available products. This is a bigger problem than ever due to the prevalence of AI systems and bots on the internet. The *Dead Internet Theory* [2], which can be traced back to 2021 states that since around 2016-2017, most of the internet is automated content created by bots. The theory has seen a resurgence in the recent years, primarily due to the rise of AI [2]. This is talked about regarding social networks, where a lot of the accounts are bots, but it can have an obvious impact in the field of reviews. If most of the internet is automated, that includes reviews, and will inherently reduce the ability of customers to trust the review system.

2.1 The influence of fake reviews

Fake reviews can have various negative effects. For example, a customer may not buy a product because of negative reviews associated with it. This is obviously unjust if there is a disinformation campaign against the product. On the other side, a product may get higher sales due to being artificially propped up by positive reviews, as a tactic of the seller to gain a market advantage over the competition. This is the main motivation behind fake reviews, visible in [Figure 1](#).

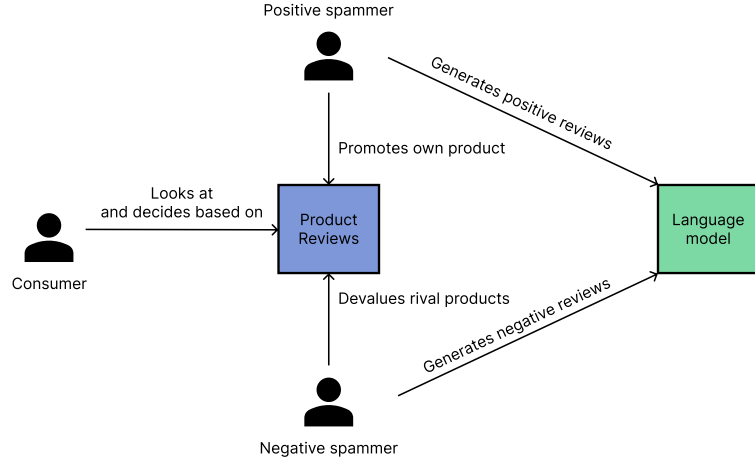


Figure 1: Spammers may promote or devalue products for market advantage. Consumers look at and decide based on them.

Influence on customer purchasing behavior

Perhaps the biggest factor in deciding whether to buy something is its reputation. That is why reviews exist, to present that information to people to better decide. The value of this is rooted in the presumed truthfulness of the reviews [15], which spammers¹ take advantage of. This doesn't only impact products ordered from the internet. [48] shows that around 93% of consumers say that online reviews influence their choice. More recently, [37] found that more than 70% of consumers read business reviews regularly when searching for local businesses and another 26% do so occasionally. Fake reviews can impact customer purchasing behavior in several ways. Firstly, a product with positive fake reviews can lead customers to buy it more, which can be harmful as fake reviews don't provide anything truthful regarding the value of the product. On the other side are negative reviews, which can cause consumers to distrust the product and also the seller [25]. Lastly, the knowledge of fake reviews existing itself can lead customers to distrust the review system itself and doubt all reviews [50], which can lower their purchasing intentions, indirectly harming the sales of online products or services.

Commercial impact of fake reviews and reviews in general

As the prior section implies, reviews can influence consumers purchasing behavior. This, in turn, can have real financial impacts on the reviewed products. Several studies show correlation between review ratings and sales. For example, [26] shows that a 1 star increase in Yelp rating leads to around a 9% increase in revenue, as an example of what may lead spammers to generate untruthful reviews. Considering only fake reviews, a 2021 study by the World Economic Forum [28] found that the impact of fake reviews on online spending is around 152 billion. They based this on the total influence of reviews, which, according to a study they cite is was 3.8 trillion in 2021 and their finding on average percentage of fake reviews on e-commerce sites, which they found to be 4%. This study also says that the eventual culling of fake reviews happens in general around after 100 days. This shows that fake reviews may even be a low-risk/high-reward situation for businesses who try to promote themselves throughout them, since the considerable gains outweigh the

¹spammer — a person who posts fake reviews [34]

comparatively low risk. More recently, [1] gathered statistics concerning fake reviews for 2025. They estimate that fake reviews cost customers more than 700 billion in 2025, that around 30% of online reviews are considered fake, and that more than 80% of customers encounter them. Interestingly, they found that the number of fake reviews grows about 12% faster than the number of other reviews, which may indicate increased AI usage for fabricating fake reviews. Another interesting thing they found is that 46% of the reviews considered fake have been positive in sentiment (5 stars). This correlates with other studies, such as [15] which also attribute overly positive sentiment as a possible indication of fake reviews.

Review bombing

Review bombing is a phenomenon that has become more and more known to the public in the last years. Most people think of it in the context of entertainment media such as video games, movies, or TV shows [16] talks about review bombing in the context of the 2020 video game *The Last of Us: Part II* and it's reviews on the review site *Metacritic*². They define review bombing as a coordinated action, where a lot of reviews are spammed on a product to alter it's evaluation. They talk about review bombing as an *ideologically driven* action, as opposed to the traditional motivations behind fake reviews (e.g. market advantage). An interesting observation they found is that the reviews follow a bimodal distribution, with spikes at very low and very high ratings, which indicates that users may try to bomb a product „in counter“ to the opposite review bombing, such as users massively spamming positive reviews to balance out the negative ones. [42] found that a sign of review bombing can be a gap between user and critic ratings. This correlates with the aforementioned study, considering that *The Last of Us: Part II* won the 2020 Game of The Year award. Both they and [16] mention some of politics, race, gender, or sexual identity as frequently discussed topics in review bombing.

2.2 Generating fake reviews using AI

In the past, fabricating fake reviews was usually done by manually writing them or buying them. Studies such as [20] show that there exists a market for fake reviews. As the cited study says, a popular communication channel for procuring fake reviews were private Facebook groups. Nowadays, malicious actors have another method of obtaining fake reviews – using generative language models. There are many options here, such as using available APIs (for example OpenAI's) for text generation, or just using a pre-trained generative model from the Hugging Face hub for review generation, or even just opening a language model web interface and asking to generate a review. [24] analyzed the frequency of AI-generated google reviews. They found that the rate nearly doubled from 2023 to 2024, and by 2024 almost one in five reviews on google is AI-generated. In addition to that, the quality of AI-generated reviews may be often higher than manually fabricated ones. [15] mentioned that humans have trouble distinguishing AI-generated reviews from real ones, often being no better than a random guesser (around 50%). Furthermore, concerning manually fabricated fake reviews, it mentions the inability of spammers to express true sentiment, which distinguishes their reviews from true ones. Large generative models whose training data includes reviews may not have this limitation. Another thing to consider is the ease of

²<https://www.metacritic.com/>

generating fake reviews. Manually writing fake reviews is time consuming, and the result is small in quantity. Just an online LLM can easily output 10 reviews in a single prompt. A python script controlling a loaded model for inference can output thousands in a tiny fraction of the time it would take to write them manually depending on the spammer's hardware.

Chapter 3

Machine learning and detecting AI-generated reviews

Detection of AI-generated reviews is a field that is still very much in development. Despite the first research papers concerning fake review detection in general (e.g. including human-written fakes) dating back years, the number of studies concerning fake reviews has largely increased, and specifically concerning AI-generated reviews. Virtually all today's studies utilize some form of Machine Learning, whether it be traditional ML models or Deep Learning models. This chapter explores the basic theory behind ML models, how their performance can be evaluated, how they can be applied for fake review detection, and then explores some of the commonly used features in the field.

3.1 Basics of Machine Learning

Broadly speaking, Machine Learning is the process of fitting an algorithm to some data, and then improvising it based on new data [33]. A Machine Learning *Model* is a generic term for a Machine Learning algorithm instantiated with parameters [49]. This model is created from training, which is the process of using an ML algorithm with data to create a model. For training, we need some important attributes about the observed samples, also called *features*, or *feature vectors* [49, 6]. For example, we could represent fake review training samples using a tuple of (*Length*, *Sentiment*, *Timeposted*). The learning algorithm would then fit the model based on the lengths, sentiments, and times posted of the reviews used for training. Machine Learning can be grouped into two main categories: *Supervised* and *Unsupervised*.

Supervised Machine Learning

Supervised ML is the process of using *labeled* data to train a model. Labeled in this case means, that in addition to the input features, the model also fits itself to *labels*, which are some form of truth about the data sample [33, 49, 6]. Concerning fake reviews, a label would be one of $\{Real, Fake\}$. Supervised learning trains models two types of tasks. The first of these is *Classification*, when the model is trained to produce *discrete* output (such as $\{Real, Fake\}$), while the second one is called *Regression*, where the model output is *continuous*. Regression is used for tasks such as forecasting or predicting something, such as future stock market prices given certain conditions (where in this case the „labels“ would be past market prices given certain conditions). [49, 6]. Since fake review detection is a

classification task, the models used in this thesis are all supervised ML models, which need a labeled dataset.

Unsupervised Machine Learning

Unsupervised ML is used primarily for *clustering*. Clustering is the process of grouping together data points by finding relation between them [49, 6, 33]. It does not directly relate to fake review detection, however it could offer additional analytics. For example, we could first classify reviews and then cluster them on additional features, and find that fake reviews are often posted at night.

3.2 Machine Learning models

This section explores some of the commonly used ML models for classification tasks and the mathematical foundation behind them. Virtually all of them can be expressed in the form of an equation with several parameters, where the parameters are specific for the classification task. The process of training the model is then fitting the equation to that task.

SVM

The information in this section is based on [43]. *Support Vector Machines (SVM)* are models, which try to separate the input features by a hyperplane. Basically, the model tries to find a hyperplane from the input features x :

$$\vec{w} \cdot \vec{x} + b = 0 \quad (3.1)$$

When using only two classes, the model then tries to map a new point to one of them using this hyperplane depending on which side of the hyperplane the new, unlabeled point lands on. If we express the classes as $\{-1, 1\}$, then the classification function f can be expressed as:

$$f(x) = \begin{cases} -1 & \text{if } \vec{w} \cdot \vec{x} + b < 0 \\ 1 & \text{if } \vec{w} \cdot \vec{x} + b \geq 0 \end{cases} \quad (3.2)$$

The concrete hyperplane the model tries to find, is the one with the largest *margin*, which is the minimal distance of the hyperplane to a point from one of the defined classes. The idea behind this is that if the margin is high, then the classes are better separated, meaning it's harder for the model to label predictions incorrectly. However, since the points may not be always separable by a hyperplane in the number of dimensions of the feature space (take for example a 2D feature space with one class concentrated in a circle, and another class in a larger circle around it). The *kernel trick* is a method of moving the feature space to a higher dimension where the classes are separable, and constructing the hyperplane there. This is done by using a *kernel function*, which computes the hyperplane in a higher dimensional space, and it does so without actually moving the samples to that space, only computing the relationship between the samples „as if they were“ in the higher dimensional space. An example of this can be visible in [Figure 2](#):

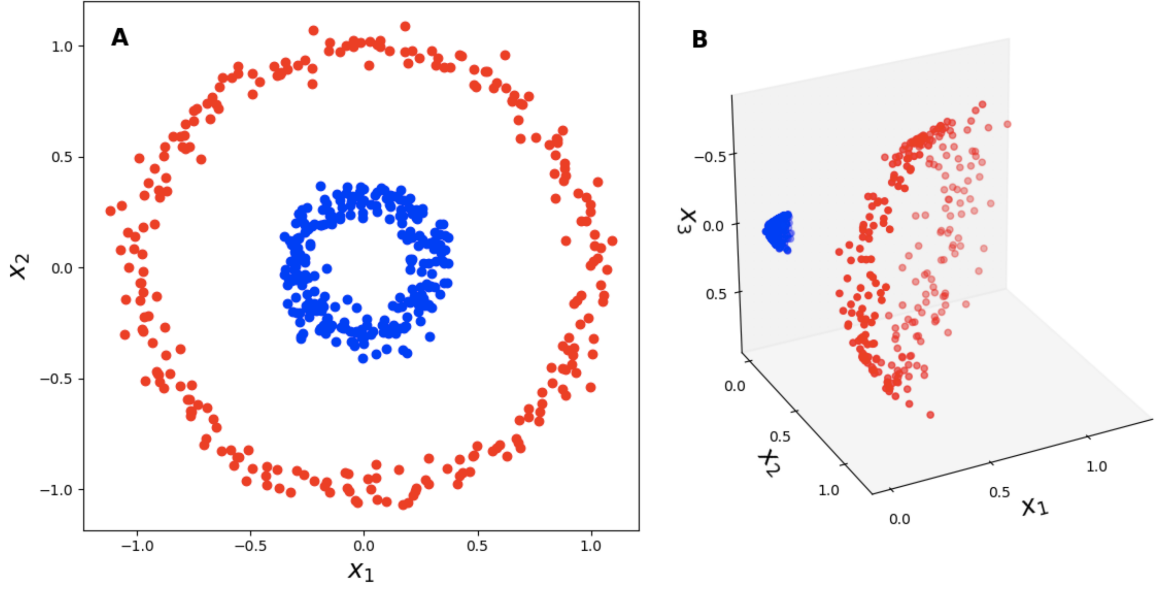


Figure 2: Input features in 2D where they aren't separable by a 2D hyperplane (a line) and in 3D, where they are separable by a 3D hyperplane (a plane) [17].

An example is the *polynomial kernel*, defined as:

$$K(\vec{x}, \vec{x}_i) = ((\vec{x}^T \vec{x}_i) + \eta)^d \quad (3.3)$$

Where $\vec{x}$ is a unlabeled example the model is trying to classify, $\vec{x}_i$ is a training example, η is a hyperparameter and d is the number of dimensions of the higher dimensional space.

Logistic Regression

Despite it's name, *Logistic Regression* is a model primarily used for classification. As opposed to normal *Linear Regression*, which predicts a *continuous* output, it predicts a *categorical* output, that being the probability (a number in $< 0, 1 >$) of the output falling into a category [38, 13]. Another misleading thing about it's name is that it is still a *linear* model. However, since the output of a linear model is continuous, and belongs in $(-\infty, \infty)$, while probabilities belong in $< 0, 1 >$ [13], the model isn't linear between the input features and the output, but instead between the input features and the *logits* of the output, which in essence means that it predicts $\text{logit}(Y)$ instead of Y from X [38, 13]. The *logit* of a variable Y is defined as:

$$\text{logit}(Y) = \ln(\text{odds}(Y)) \quad (3.4)$$

where odds are the probability of Y happening, e.g. $p(Y)$ relative to the probability of Y not happening, e.g. $1 - p(Y)$. Then the equation can be written as:

$$\text{logit}(Y) = \ln\left(\frac{p(Y)}{1 - p(Y)}\right) \quad (3.5)$$

and the model tries to equate this to a linear model, meaning it tries to solve:

$$\ln\left(\frac{p(Y)}{1 - p(Y)}\right) = \alpha + \beta X \quad (3.6)$$

Which, after exponentiation and simplifying, results in the following [38, 13]:

$$p(Y|X = x) = \frac{e^{\alpha + \beta X}}{1 + e^{\alpha + \beta X}} \quad (3.7)$$

This function always outputs a number between 0 and 1. Classification done by a threshold, usually 0.5 [13]. For example, if the probability of a review being fake exceeds 0.5, we classify it as fake, otherwise, we classify it as authentic.

Decision Trees and Random Forests

The information in this section is based on [29]. Decision trees are models, which continuously split the data based on values of different features until a stopping criterion is reached. This builds a tree like structure, where the root node represents the entire dataset, and the leaf nodes are individual class labels. An example of a decision tree algorithm is ID3 (*Iterative Dichotomiser 3*), which chooses the feature to split on based on information gain. Also, one of the biggest advantages of decision trees are that they are easy to visualize and interpret, for example when evaluating feature importance, as opposed to for example Deep Learning models. *Random Forests* are models built from several decision trees. These trees have random subsets from the dataset and a random subset of features. This can improve vulnerabilities that normal decision trees are prone to, such as overfitting. Figure 3 shows a overview of the Random Forest model.

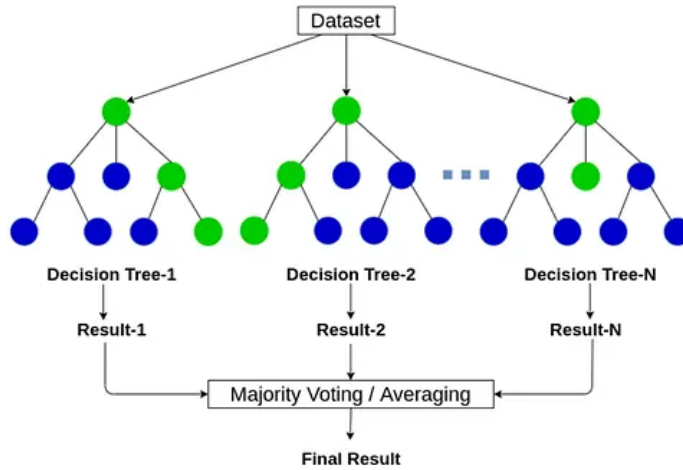


Figure 3: High level overview of Random Forests: The predictions of the trees are transformed into one prediction via majority voting or averaging [22].

The predictions of the individual trees are then transformed into one prediction via majority voting or averaging [22].

Naive Bayes

The *Naive Bayes* classifiers are a family of models based on the *Bayes Theorem*, which is a way of calculating a conditional probability of a event given another event. It uses the reverse conditional probability and the independent probabilities of the two events. It is defined as [8]:

$$P(B | A) = \frac{P(A | B) \cdot P(B)}{P(A)} \quad (3.8)$$

When using this for classification tasks the model tries to calculate the probability of a given class, given the input data (features) [8]. For example, for AI generated review detection, for the review „*I hate this hotel!*“ we are trying to find $P(\text{AI-generated} | \text{I hate this hotel})$, meaning the probability the review is AI-generated, given that it is „*I hate this hotel!*“. The model makes the *naive* assumption that all input variables are independent of each other. For text classification, if we take words/unigrams as features, this assumption means that the word orders are independent of each other [41]. An implication of this is that the model only considers word frequencies, and the review „*The hotel was good, but the food was terrible!*“ will be always classified the same as „*The hotel was terrible, but the food was good!*“ despite one probably having a higher rating than the other. However, this massively simplifies the computation, as it means that the overall probability can be split into a product of probabilities of each feature [8]. In this example, that would mean that:

$$P(\text{AI-generated} | \text{I hate this hotel!}) = P(\text{AI-generated} | \text{I}) * \dots * P(\text{AI-generated} | \text{hotel}) \quad (3.9)$$

While the classifier has multiple versions, the one which considers a document as a sequence of words and uses word frequencies is called *Multinomial Naive Bayes* [41].

KNN

The information from this subsection is based on [10]. The *K-Nearest-Neighbors* classifier is very simplistic in it's core idea. It classifies a new sample based on it's closest neighbors among the training samples, and assigns it to their class. The variants of the classifier differ in three things:

1. The K in KNN, e.g. the number of neighbors used
2. The distance metric used to compute the nearest neighbors
3. The voting function, which determines the class of the new sample using the nearest neighbors

Common distance metrics include variants of the *Minkowski* distance, which for N features, that is $N - \text{dimensional}$ spaces is defined as:

$$d(y, x_i) = \left(\sum_{n=1}^N |y_n - x_{in}|^p \right)^{\frac{1}{p}} \quad (3.10)$$

And is most often used with $p = 1$, $p = 2$ or $p = \infty$ which are the Manhattan, Euclidean and Chebyshev distances respectively. For the voting function, a common one is to use a weighted one where the closest neighbors' votes weigh more than other.

Ensemble Models

The information in this section is based on [30, 23]. Some classifiers naturally have their strengths and weaknesses. Ensemble Models, or Ensemble Classifiers are classifiers which

are composed of multiple classifiers, and their prediction are, in some way, combined to make one final prediction, which aims to outperform the predictions of individual models by combining their strengths. The simplest form of an ensemble model is combining the outputs of the individual classifiers via voting, or weighted majority voting. The three most common approaches for building ensembles are called *Bagging*, *Boosting*, and *Stacking*.

Bagging, or *Bootstrap Aggregating* is a method which builds an ensemble via building N classifiers on N random subsets of data. It shows the best improvements when the base classifier is unstable, which means that it changes its generalization ability significantly on slight modifications to its input. The final output of a Bagging algorithm is decided using majority voting. An example of a bagging algorithm is the aforementioned Random Forest.

Boosting is a method based on the idea that if we have a model which doesn't perform well, we can iteratively convert it to a strong model using a set of weaker models. It works by training a base model, looking at which samples it got wrong, and training the next model on those samples. It also increases the weights of the misclassified samples so that the algorithm focuses on them more. It also assigns bigger weights to classifiers that had better performance during training, and decides the final output with weighted majority voting.

Stacking is different from Bagging and Boosting mainly in that it trains several models with different algorithms on the same dataset. It works by first training several base models, also called level 0 models on the dataset. The predictions of these models are used as the new dataset, also called the meta dataset. Then, another model, called the meta learner, or the level 1 model is trained on this dataset. Its prediction is then taken as the output of the ensemble.

Per recent surveys [34, 18], Ensemble models can improve the performance of fake review classification. They mention several studies which chose an ensemble approach, for example based on review and reviewer features.

Neural networks

Neural networks are the foundation of all deep learning, including more complicated models like modern LLMs. They are rooted in biological inspiration, particularly in biological brains and biological neurons [44, 21]. They are composed of several layers, which are interconnected between themselves. The first layer is called the *input* layer, the final one is called the *output* layers, and the layers in between are called the *hidden* layers [44]. Each layer is composed of one or more artificial neurons, where each neuron is connected to all the neurons of the layers next to it. This neuron takes in several inputs, and produces an output. The inputs include [44, 21]:

1. The input data x_i . This can be the output of the i_{th} neuron of the previous layer, or the unprocessed input to the whole network.
2. Weights w_i . These decide how much a particular input is „important“ to that neuron.
3. Bias b .

The output of the neuron is based on the dot product of the vectors w and x with b added, computed as [44]:

$$dot = \vec{x} \cdot \vec{w} + b \quad (3.11)$$

This number is passed through an *activation function*, which decides the output of the neuron, where the most common one is the *ReLU*, short for *Rectified Linear Unit*, computed as [44]:

$$ReLU(x) = \max(0, x) \quad (3.12)$$

Figure 4 displays the architecture of a Neural Network.

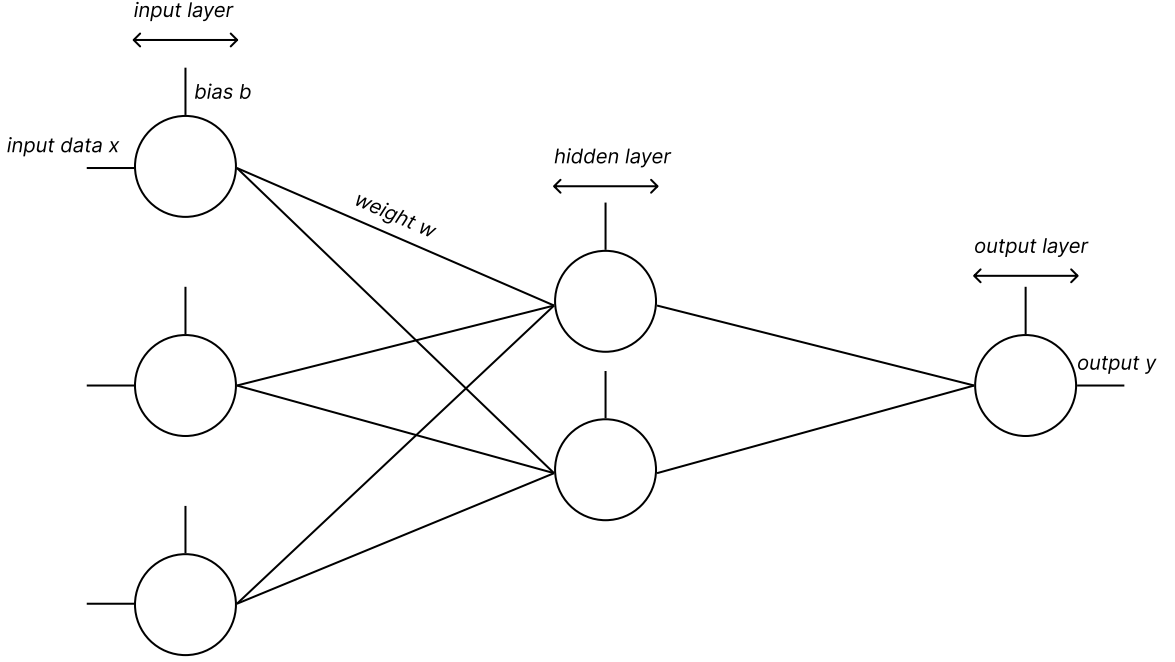


Figure 4: Example of a neural network. Figure heavily inspired by a figure from [44].

Transformers based language models

Modern large language models are based on the transformers architecture, which was introduced in the 2017 paper *Attention is all you need* [47]. What makes these models so special is the *self-attention* mechanism, which allows a model to add context to word embeddings based on the words around it [4], which solves the problem of the embeddings being static for each word. The original architecture is composed of encoder and decoder parts, where the encoder is responsible for contextualizing the input embedding and the decoder takes it as an input and generates the next word [47]. Both the Encoder and Decoder are a stack of identical layers, where each Encoder layer is composed of self-attention followed by a feed-forward neural network, while the Decoder is similar, but it masks future positions in the self-attention part to only make the new word depend on past ones, and it adds a *Encoder-Decoder attention* layer into the middle, which performs self-attention on the output of the final Encoder [47, 4]. However, most modern models only have one of these components. *Encoder* models, with only the Encoder component are often used for tasks such as classification and sentiment analysis, since they can create contextualized word embeddings [12]. The best-known model in this lineage is *BERT*, introduced in 2018 [12].

Decoder models, on the other hand, are used for generative tasks. The best known Decoder only model is the *GPT* model [39], also introduced in 2018, which is the model whose newer version is behind the *Chat-GPT* web chat interface. While Encoder model usage would be expected for fake review classification, [15] also used *GPT-Neo*, a Decoder model to classify fake reviews with good results. However, for example [33] did use a Encoder model for creating embeddings, and then using those as features for their other classifiers.

3.3 Model evaluation metrics

This section summarizes the most commonly used metrics for evaluating classification. The information in it is based on [40]. When classifying fake reviews, we have 2 classes: $\{Fake, Real\}$. The following metrics work with these variables, described in the context of fake review detection:

- *False Positives (FP)*: Number of reviews incorrectly classified as fake.
- *True Positives (TP)*: Number of reviews correctly classified as fake.
- *False Negatives (FN)*: Number of reviews incorrectly classified as authentic.
- *True Negatives (TN)*: Number of reviews correctly classified as authentic.

Accuracy is a metric which provides a general overview of how often the model was right as opposed to how often it was wrong. It is defined as:

$$ACC = \frac{TP + TN}{FP + FN + TP + TN} \quad (3.13)$$

The biggest problem with Accuracy is the case of imbalanced datasets. If we have 99 authentic reviews and 1 fake review, a model which always predicts fake could reach Accuracy of 99% by just guessing *Real* every time.

Precision is a metric describing that how often was the model right, if it guessed the predicted class. In this case, it would mean that out of all reviews the model classified as fake, how often it was right. It is defined as:

$$PRE = \frac{TP}{TP + FP} \quad (3.14)$$

Recall describes how many of the predicted class did the model „find“. In this case, it describes that out of all fake reviews, how many did the model find. It is defined as:

$$REC = \frac{TP}{TP + FN} \quad (3.15)$$

Precision and Recall focus on different „priorities“. Precision is important when a big *FP* could be a problem, while Recall is important when we want to find as much of the predicted class as possible. *F1-Score* combines both Precision and Recall. It is defined as:

$$F1 = 2 \cdot \frac{PRE \cdot REC}{PRE + REC} \quad (3.16)$$

3.4 Commonly used features in fake review detection

This section explores some of the most commonly used features in fake review detection, and text classification overall. They include stylometric features, features that focus on word counts such as TF-IDF or Ngrams, semantic features like sentiment and embedding and review specific features such as product or profile metadata, or rating of the review.

Stylometric features

Stylometry is the study of writing style. Several studies utilize it for feature extraction, since language models seem to have different writing style than humans. Some of the most common stylometric features are the frequency of using punctuation, average sentence or word length, n-gram features, or how often the review repeats words ([34], [18], [36]). Parts of speech can also be used as stylometric features. They can be used in various ways, such as the frequency of various POS in a given review [34], or their count [36].

Concerning unique words, one of the best known features concerning lexical diversity is the *Type-Token Ratio (TTR)* [27]. It is defined for a text T and individual tokens t as:

$$TTR(T) = \frac{|\{t \mid t \in T\}|}{|T|} \quad (3.17)$$

It's a number inside $< 0, 1 >$ indicating how lexically diverse the input text is. It could be useful to detect a model using a limited set of words.

Hapax-Legomenon rate is a feature that could also evaluate the lexical diversity of a text [36]. It is defined for a text T and tokens t as:

$$HLR(T) = \frac{|\{t \mid count(t, T) = 1\}|}{|T|} \quad (3.18)$$

Interestingly enough, it was one of the 4 most important features in [36]. It could detect the usage of uncommon words, but also for example a model with parameters that punish repetition.

Another useful metric which measures the simplicity of a input text is the *Automated Readability Index (ARI)*. Parts of the following paragraph about it are paraphrased from [15]. For a given text, ARI is defined as:

$$ARI(text) = 4.71 \frac{|c \mid c \in text \wedge ischar(c)|}{|w \mid w \in text \wedge isword(w)|} + 0.5 \frac{|w \mid w \in text \wedge isword(w)|}{|s \mid s \in text \wedge issentence(s)|} - 21.43 \quad (3.19)$$

ARI is a readability metric which indicates the US education level needed to understand the text. A score around 3 means that a US third grader can understand it, etc. It is similar to TTR in that a higher number usually indicates a more complex text.

N-grams are sequences of elements of length N . „Elements“ most usually refers to words here, but there can be character or token based N-grams [3]. Unigrams have 1 element, bigrams have 2, and so on. For example, if we generalize elements to words, and ask which bigrams are in the sentence „*Cats are very cute*“, they would be:

$$\{(Cats, are), (are, very), (very, cute)\}$$

N-grams can be useful features for finding out which phrases are the most common in the input text. For example, if a language model tends to repeat the same phrases often, the n-grams corresponding to these phrases will appear more often than other ones. This could be especially useful in something as specific as fake review detection, where the models used to generate spam could tend to repeat the same positive or negative phrases.

TF-IDF (*Term Frequency-Inverse Document Frequency*) is a feature extraction method. It most usually works with individual words, but it can be character level or n-gram level. The individual elements are called *terms*. Parts of the following paragraph are taken from [11]. To compute the vector representation of the input text, TF-IDF uses two vectors. For a term t and a dataset entry E , the TF vector is defined as:

$$TF(t, E) = \frac{count(t, E)}{|E|} \quad (3.20)$$

In simple terms, it's a score which is indicating how relevant t is in E . The more often it appears, the higher the TF score will be.

The second vector, IDF is composed of elements defined for a term t as:

$$IDF(t) = \log\left(\frac{|D|}{|\{E \in D \mid t \in E\}|}\right) \quad (3.21)$$

This vector assigns a higher score to terms which appear in less documents, hence the *inverse*. As opposed to TF, this vector is mutual for the entire dataset, and as such does not need to be computed for every text inside.

The resulting vectorized text, TF-IDF is then defined as:

$$TF-IDF(t, E) = TF(t, E) \cdot IDF(t) \quad (3.22)$$

meaning that the highest score terms are the ones, which are relevant to that specific entry but don't appear too often in other entries.

Semantic features

Semantic features focus on the meaning of the text instead of it's structure. This can allow them to model relationships between words that aren't seen just from the structure, or extract deeper meaning from the review.

A common semantic feature is the *sentiment* of the review. Per multiple studies ([46], [19], [15]) polarized sentiment is prevalent in fake reviews. That means that fake reviews tend to be very positive or very negative, but very few „in the middle“. Per [19] however, positive fake reviews outnumber negative fake reviews (which might be expected, since promoting own product seems like it would have more influence than defaming the competition). Review sentiment can be extracted in various ways, such as lexicon based tools [46] or by using a sentiment classifier.

Word embeddings are representations of words as dense, continuous vectors, where each element of the vector represents some property of that word [31, 5]. They were made popular by the 2013 paper *Word2vec* [31], where they created these vectors using neural networks which for a given word, tried to predict the words it was most likely to appear next to. Since these representations are vectors, they allow us to perform operations like addition or subtraction of different words, or finding the most similar words via techniques like cosine similarity [31]. A classic example is that if we subtract the vector for „Man“ from the vector for „King“ and add the vector for „Woman“, we will get a vector that is very close to the word „Queen“ [31, 5]. The underlying features and what they represent are more of a black box to us, since we don’t know what the individual dimensions in the embedding vector space represent, but they are a very effective method to turn text into numbers and capture more than it’s structure. Word embeddings are primarily used as features with deep learning models (various neural network architectures, Transformer-based models) but can be used as features with other classifiers as well. For example, [33] used a BERT model to extract contextualized word embeddings, and then trained multiple classifiers on these embeddings, such as SVM, Random Forest, and more.

Review and product metadata

Review metadata are features about the review, but not about it’s text. They go hand in hand with metadata about the product. Examples of this include review rating, product ID, or date and time posted ([34], [18]). One of the aforementioned studies ([15]) which talks about the polarized sentiment of fake reviews also found that fake reviews tend to follow a bimodal rating distribution, meaning they have sharp peaks at the lowest and highest rating. Other studies have found that other ratings of the product could also be a useful feature, with [45] finding that the number of 1 star ratings on a product was one of the most important features in their Random Forest model.

Reviewer centric features

Reviewer centric features are characteristics of the reviewer’s profile. [34] lists some of these features. They include features such as repeated reviews across products reviewed, sentiment of the reviews (for a concrete reviewer), or burstiness (tendency to post a lot of reviews in a short time span). Another survey [18] also lists the absence of profile data, or review time span. These features can greatly improve the results of the detection [34, 18], however they are difficult to obtain.

Chapter 4

The Booking.com synthetic review dataset

The dataset used in this thesis was collected using reviews of Slovak hotels on [Booking.com](https://www.booking.com). From the scraped reviews, two variants were created. One where each review was used as „unified“ content (meaning the positive and negative content were concatenated) and one where they were used as separate samples.

4.1 Choice of the data source

Part of the reason why Booking.com was chosen is due to inspiration by previous datasets used in fake review detection. The hospitality sector dominates the percentage of the used datasets, per [34] and [18]. [Figure 5](#), taken from [18] and recreated into a SVG shows the distribution of the datasets used from 2021 to 2023:

Domains in fake review detection datasets from 2021 to 2023

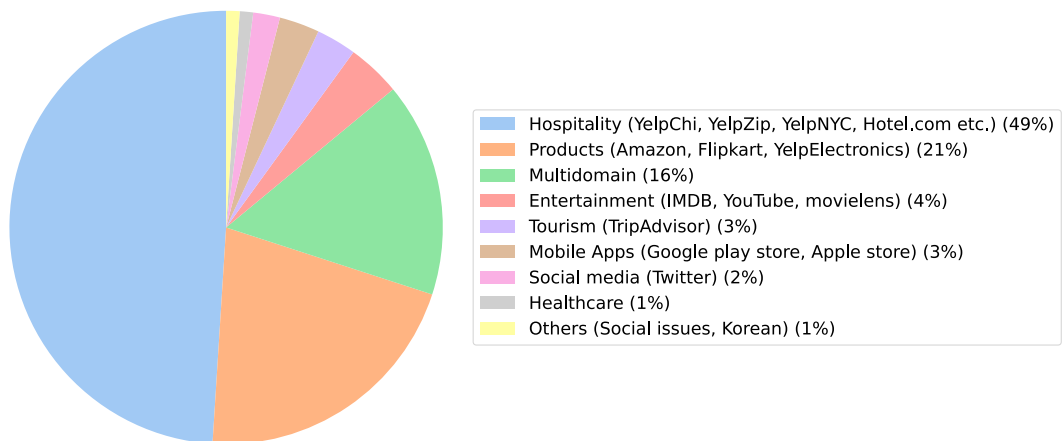


Figure 5: Distribution of dataset domains used in fake review detection from 2021 to 2023

To my best knowledge, there is no dataset of synthetic reviews in the Slovak language. There do exist some review datasets in Slovak, however none of them match the scale and topics of review datasets in other languages:

1. **STS2-sk**¹: Dataset of reviews across 7 different categories (accommodation, books, cars, games, mobiles, movies, and stress). However, has a very small amount of samples to train a model on. It only includes about 100 reviews for each category, and only 89 for hotel reviews.
2. **ČSFD Movie Reviews**²: Contains 25 000 movie reviews. However, fake movie (and entertainment in general) reviews may look different to fake reviews on customer domains, as the motivation behind them is often different (such as review bombing, as opposed to fake reviews traditionally being motivated by profit).

Booking.com represents the largest source of reviews of hotels all over Slovakia, with options to filter by language to use only reviews in Slovak. The website can be easily parsed, and the reviews are structured into 2 parts (the „good part“, or „what did you like?“, and the „bad part“, or „what did you dislike?“) which is more complicated than 1 review = 1 text, but allows for multiple variants (use a review as one sample or as 2 samples).

4.2 Collecting data

A total of 10064 reviews were scraped from the Booking.com website. The scraper always navigated to the website and searched for hotels in Slovakia. Then it would go hotel by hotel, filter the reviews by language to Slovak, and scrape each review from a hotel which it didn't scrape already (known by URL). The scraper, which was a headless browser implemented in Python using Playwright would extract the hotel name (the individual CSV files were saved by their hotel name for filtering already scraped hotels), the positive content, negative content and rating and save them into a CSV.

Scraping permission and legal aspects of the used data

Since web scraping is a legally gray area, and Booking.com's terms and conditions³ disallow any sort of monitoring, scraping and crawling, permission was asked from their data privacy office. They granted it under the conditions that the collected data is anonymized, the review content is not referred to, and the data is not shared. Per their request, the dataset does not contain any personal data, and it's content is not published other than aggregate statistics.

4.3 Dataset characteristics

This section describes the non synthetic part of the dataset, it's characteristics and statistics. The dataset contains 10064 rows, each of which can be defined as a quadruple of (*rating*, *content_{good}*, *content_{bad}*, *content_{unified}*) where the last is just the two content parts concatenated with a whitespace between them if both are present (else it's identical to the present part). **Table 1** describes how many of the scraped reviews contain only positive and negative content, and how many contain both.

¹https://github.com/kinit-sk/slovakbert-auxiliary/tree/main/sentiment_reviews

²https://huggingface.co/datasets/fewshot-goes-multilingual/sk_csfd-movie-reviews

³<https://www.booking.com/content/terms>

Content type	Count
Total reviews	10064
Only positive content	2587
Only negative content	370
Both positive and negative content	7107

Table 1: Statistic of content type presence in the reviews.

From the table we can see that about three quarters of the dataset have both negative and positive content. Out of those that don't, there are almost 7 times as many reviews with only positive content than reviews with only negative content. From this, it comes as no surprise that the dataset heavily skews towards positive ratings. Booking.com uses ratings in the interval $< 1, 10 >$ and Figure 6 shows that the mean rating is 8.24, and if we go by discrete bins, reviews with a rating of 10 are by far the most common. This might be due to Booking.com pushing highly rated hotels up as default results in a location.

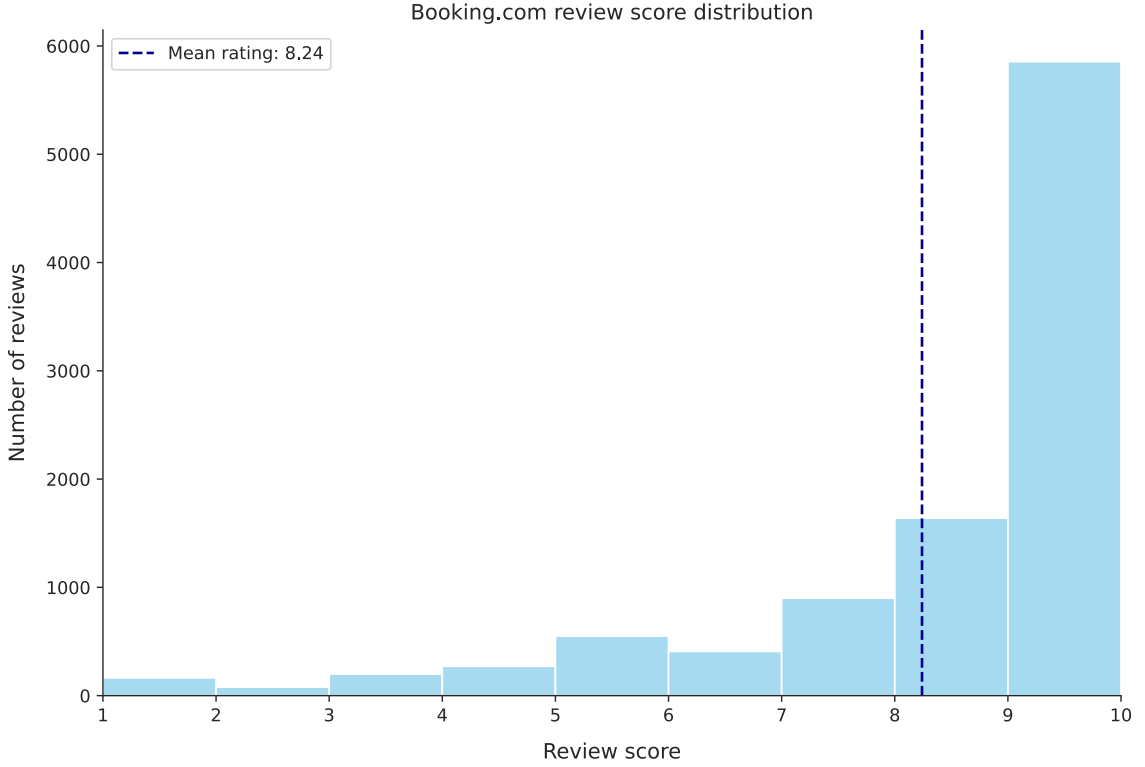


Figure 6: Distribution of ratings in the dataset.

When it comes to the text length, Figure 7 and Table 8 show that both the positive and negative parts of the reviews are mostly around 110 characters (in median), with long outliers skewing the mean to the right, but both parts don't significantly differ from each other.

Content type	Average length	Median length	Max length	Min length
Positive	148.65	117.00	1988.0	1.0
Negative	159.18	106.00	1998.0	1.0
Total	262.15	195.00	2810	96

Table 2: Statistics of review lengths for different content types.

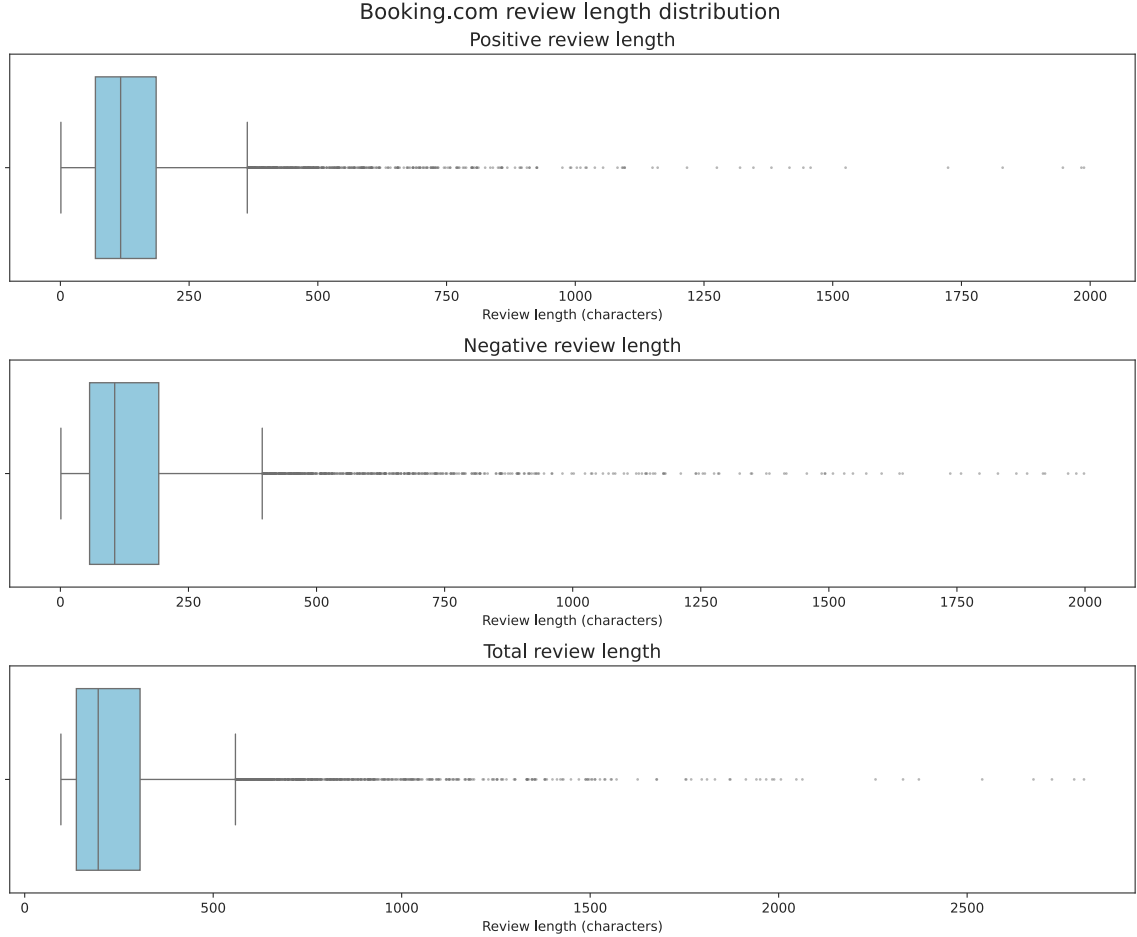


Figure 7: Distribution of review lengths in the dataset.

Data cleaning

The cleaning of scraped reviews was done primarily on their length. First, duplicate and blank reviews were dropped. Then, reviews which were labeled as outliers due to their length (in words) were dropped. The range for the review lengths to keep was $< Q1, Q3 + 1.5 * IQR >$, where $Q1$ and $Q3$ are the first and third quartile of review lengths respectively, while IQR is $Q3 - Q1$. Usually the left side of the non-outlier set is $Q1 - 1.5 * IQR$ [14], however since that went into the negative for this dataset, and the start of $Q1$ were reasonably long reviews for both variants, $Q1$ was used as the left bound.

4.4 Generating synthetic data

Datasets used in the past have been constructed by methods such as crowdsourcing [34] or some other form of asking people to manually fabricate fake reviews. Another commonly used method was by using a filtering algorithm. One of the oldest datasets for fake review detection [35] collected hotel reviews from Yelp, which has its own filtering algorithm but doesn't delete reviews labeled as spam, but puts them in a separate list. However, the first method makes it difficult to gather large amounts of data, while the second labels reviews based on a non transparent algorithm. Another option is to create a new, synthetic dataset based on the „real“ one which contains scraped reviews. This involves using a generative language model to generate reviews and annotating them as fake. [15] which used GPT-3 and [32] which used various models have found success with this method. Its benefit is that it allows the generation of high quality annotated data very quickly, however we still need the „real“ part of the dataset.

Model comparison

For generating synthetic reviews, 3 models were evaluated. Two of them were instruct models. The first was a pre-quantized (into 4bit by Unsloth) 27b version of the Gemma 3 model ⁴ by Google, and the second was a 32b version of the Qwen 3 model ⁵ by Qwen. Both of those are known for their multilingual capabilities. The third model was the mistral-sk-7b model ⁶, which is a fine-tuned mistral model for the Slovak language. This model is a base model. Each of these models was used to generate 200 samples from the unified variant of the dataset, and 200 samples from the separate. Then the generated text quality was compared with 2 methods:

1. Perplexity: For each of the generated sample „datasets“, perplexity was computed on the whole set using an independent model.
2. Survey: A Google Forms survey was created, asking humans to rank the models by how „natural“ the text is.

Model prompts

The generative models were always prompted with example reviews from the dataset. To make the synthetic data follow the sentiment distribution of the real dataset, the examples were sampled as follows. One review was always randomly sampled from the entire dataset. Then:

1. For the separate variant: 5 examples were randomly chosen from the part of the dataset that matched its content type (positive /negative). The content type was stored alongside the new rating for later stratification into training/validation/testing sets.

⁴<https://huggingface.co/unsloth/gemma-3-27b-it-bnb-4bit>

⁵<https://huggingface.co/Qwen/Qwen3-32B>

⁶<https://huggingface.co/slovak-nlp/mistral-sk-7b>

2. For the unified variant: 5 examples were randomly chosen from the part of the dataset whose rating was in $< \max(\text{rating}_{\text{example}} - 1, 1), \min(\text{rating}_{\text{example}} + 1, 10) >$. The rating of the example was also stored alongside the new review.

The prompts itself however differed between models. Since the mistral model is not a instruct model, which means it's not tuned to follow instructions, we couldn't use a prompt like „Vygeneruj recenziu hotela po slovensky...“. Instead, the model was always left to fill in the last example. The generic prompt for this model then looked like:

```

Hotelová recenzia 1: <Example 1>
Hotelová recenzia 2: <Example 2>
...
Hotelová recenzia 6:

```

On the other hand, the instruct models allowed bigger flexibility with the prompt. Based on the sampled content type of rating, the prompt was always a bit different. The models followed a label instruction, which described what should the review's sentiment look like. This is showcased in tables [Table ??](#) and `tab:separate_label_instruction_for_the_unified_and_separate_variant`.

Rating	Label instruction
≥ 8	veľmi spokojná
≥ 6	spokojná
≥ 3	nespokojná
< 3	veľmi nespokojná

Table 3: The label instructions based on the sampled rating.

Content type	Label instruction
$content_{\text{good}}$	o tom čo sa hosťom páčilo
$content_{\text{bad}}$	o tom čo sa hosťom nepáčilo

Table 4: The label instructions based on the sampled content type.

The models also produced much longer reviews than the given examples. To mitigate this, the min review word count, max review word count and average review word count were computed from the examples and included in the prompt, as:

```

f"Dĺžka má byť približne {target_words} slov "
f"(rozsah {min_words}-{max_words} slov). "
f"Nepíš viac ako {max_words} slov.

```

Model evaluation

To evaluate the model outputs and choose the best one to use in the dataset, the following was done:

1. Their Perplexity was computed using an independent model and compared
2. A Google Forms survey with their outputs was created

Perplexity

Perplexity (*PPL*) is a generative LLM evaluation metric, which describes how much likely a word is to follow it’s preceding ones. A lower perplexity score usually implies better-quality and more coherent text. It is defined as [15]:

$$PPL(X) = \exp(-\frac{1}{t} \sum_{i=1}^t \log p(x_i | x_{<i})) \quad (4.1)$$

The perplexity was computed for each text in the sample datasets, summed up and then averaged. To avoid a situation where a model would assign itself a lower perplexity score, a independent model was used to compute this. The model used was the EuroLLM-22B-Instruct-2512 model ⁷, which has Slovak in the list of languages it understands. The results are visible in Table ??.

	Model	Perplexity	Variant
0	Gemma	7.107292	Unified
1	Qwen	17.466667	Unified
2	Mistral	17.810422	Unified
3	Gemma	14.902512	Separate
4	Qwen	73.536782	Separate
5	Mistral	47.902919	Separate

Table 5: Perplexity scores for the individual models for each variant.

For both variants, the Gemma model has the best score. The other two models are close in the unified variant, but the mistral model is a distant second in the separate one.

Google Forms survey

The other technique used to evaluate the generated text was to let human evaluators read it and rank the models. A Google Forms survey was created with 10 questions. Each question contained one sample from each model, where 5 of the questions contained samples from the unified variant and 5 from the separate. The evaluators were asked to rank the examples from most to least natural sounding, e.g. to rank the one where they would most likely believe that it was human written at the top, and the opposite at the bottom. Each model has assigned a point for a first place vote, half of a point for a second place vote, and zero points for a third place vote. So the total score the model received was:

$$Score = votes_{first} + \frac{votes_{second}}{2} \quad (4.2)$$

The results are visible in Table ??.

⁷<https://huggingface.co/utter-project/EuroLLM-22B-Instruct-2512>

	First place votes	Second place votes	Third place votes	Total score
Gemma	39	25	46	51.5
Mistral	49	36	25	67
Qwen	22	49	39	46.5

Table 6: Results of the Google Forms survey.

As opposed to perplexity, the Mistral model wins this by a sizable margin, with the second (Gemma) and third (Qwen) model being pretty close to each other.

Model choice

In the end, the Gemma model was chosen instead of the Mistral model. The Qwen model was disqualified by placing last in both metrics. The Gemma model was chosen due to several reasons:

1. Larger model: The Gemma model is more than 4 times larger than the Mistral model. This reflects in the generated texts where this model is able to be semantically richer and generate longer reviews which are still in coherent Slovak.
2. Instruct model: This makes tweaking the synthetic data to match the real data much easier, since things like expected length or sentiment can be inserted into the prompt.

Initial results of generating data

The initial synthetic half of the dataset was generated using the Gemma 3 model. However, while the reviews initially seemed good enough, they were very easily detectable with a simple model in a bigger sample size. A simple SVM model using TF-IDF as features immediately reached almost 100% accuracy and F1-score on the review dataset. Reasons for this were mostly the model using different wording despite being given examples than the dataset, it’s tendency to repeat itself/talk about the same topics. Another reason was that it’s sentiment was visibly much more polarized than the original dataset. For example, the model used exclamation marks in almost every single review, which could serve as a shortcut for the classifier to immediately detect it. For this reason, the Gemma 3 model was additionally fine-tuned on the review dataset.

Fine-tuning Gemma 3

A fine-tuning version of the dataset was created for both it’s variants. The process was done as follows:

1. For each review, a instruct-response pair was created. The response was the sampled review, while the instruction was the original prompt for Gemma 3 with the review length/sentiment and a few tweaks, such as the rating label instructions being less polarized in sentiment (pozitívna, skôr pozitívna, skôr negatívna, negatívna).
2. The model was trained on the fine-tuning dataset and saved. The training parameters used are in Appendix 1.

The entire process of the final dataset creation can be visible in [Figure 8](#). The reviews are scraped, Gemma 3 is fine-tuned on them to generate synthetic ones and they are combined with real ones to create the review dataset.

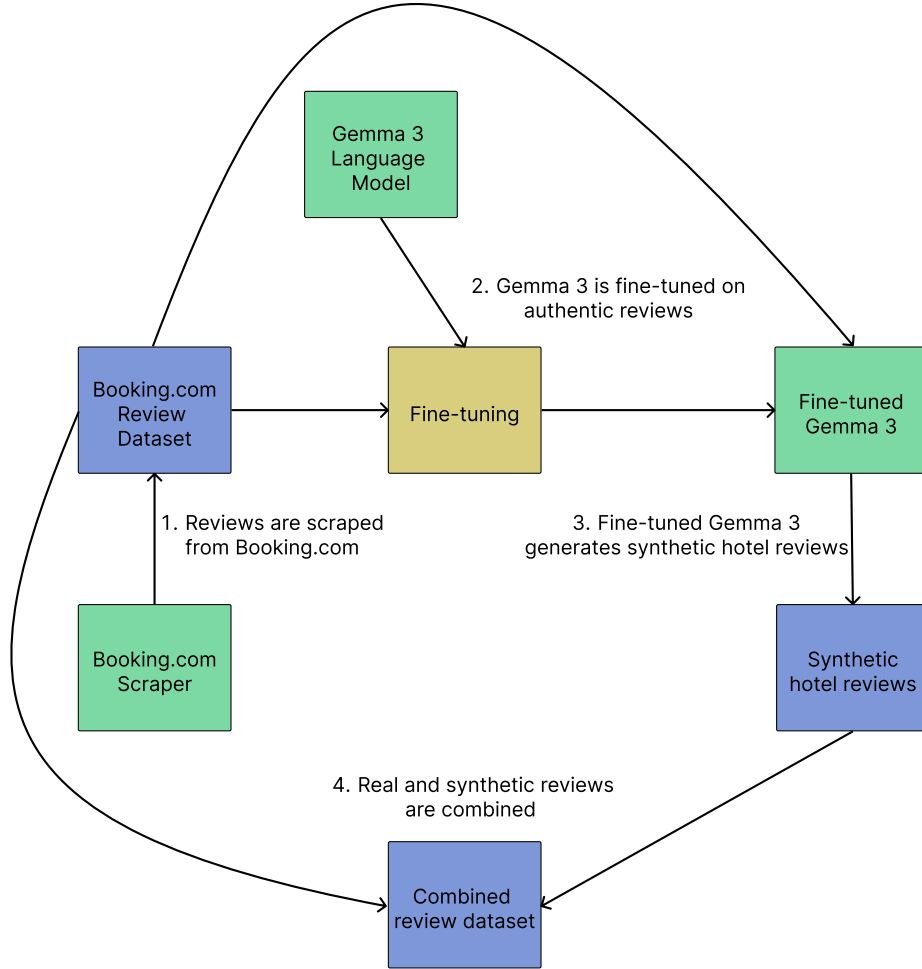


Figure 8: Process of creating the review dataset.

Total number of reviews in the dataset

[Table 7](#) shows the final amount of reviews inside the dataset for both of it's variants:

Variant	AI-Generated	Authentic	Total
Separate	11700	11756	23456
Unified	7000	6999	13999

Table 7: Dataset counts for separate and unified variants

4.5 Splitting the dataset for training and testing

Good splits of the data into training, validation and test splits are important to avoid problems during model training such as overfitting. They depend on the size of the dataset,

and how balanced it is among other factors. A common ratio is splitting it into 80% training, and 10% per the validation and test splits [7]. Another common method is to use *K-Fold cross validation* [7], which splits the dataset into K folds. Then for each of these folds (so K times), it trains the model on the other $K - 1$ folds and validates it on the current fold. This ensures we use all of the data for both training and validation. Per the cited article, there are multiple techniques to choose „which data goes into each set“. If the dataset is balanced, meaning there isn’t one group of the data larger than the other random sampling should suffice, however if the sets aren’t equal, stratified sampling might be better. That means that the percentage of the data in each set for a given group corresponds to the total percentage of the group in the dataset. The dataset is balanced when it comes to the volume of real and synthetic data, however it’s not balanced when it comes to sentiment. For this reason, the data is sampled using the stratified strategy. As mentioned in [Section 4.4](#), the chosen rating/content type was stored alongside the generated review. Then, when creating the training, validation and test splits, they were used as variable for their variant respectively alongside the class label (whether the review is real or synthetic) for stratified sampling. Concerning the ratio of the splits, the dataset was always randomly split using different strategies. The first split was a 90-10 train/test split, used for evaluating individual models. The second was a 80/10/10 train/validation/test split used to find best hyperparameters of the base models, and the third one was a 90/10 train test split with the 90% training set being equally divided into 5 folds for K-Fold cross validation, used for the full ensemble training, meta dataset generation, and finding the optimal parameters for the meta model.

Chapter 5

Architecture of the classifier

For the classifier, a stacked ensemble model was chosen with 3 inner models. Each of the models focuses on different features of the dataset, and their outputs are fed as inputs to the meta learner, who makes the final prediction. **Figure 9** displays a high level overview of the proposed architecture.

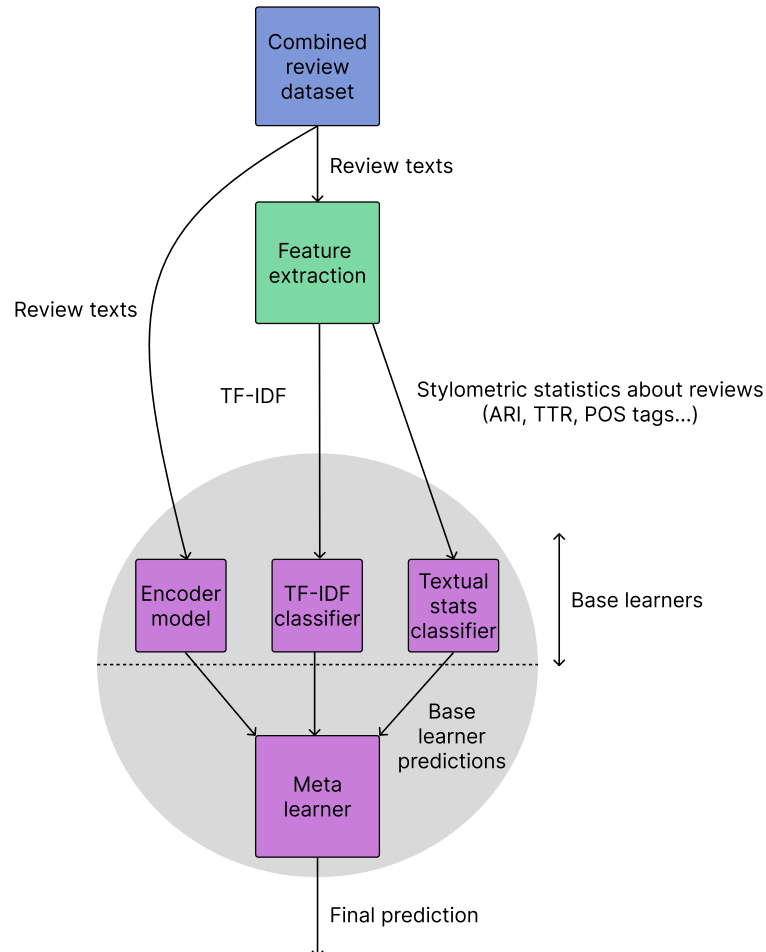


Figure 9: Architecture of the fake review detector.

The base learners are trained on either the raw review dataset, or vectors of features which are extracted from it to be used as input to the model. In the first stage, the base learners create a *meta dataset*, which contains probabilities of the review being fake for each model. Then the meta learner is trained on these probabilities, and it itself outputs a probability of the review being fake.

5.1 Model components

The statistical classifier

This part of the ensemble focuses on stylometric statistics extracted from the input text. It is trained on vectors of scalar values, where each value represents a different feature of the input review. It uses the `StatisticalFeatureExtractor` class, which runs each feature extraction in parallel using Python's `multiprocessing` module to create a process pool. The result is a dense feature matrix of size $(n_{samples}, n_{features})$ which is fed to the model. All input features are standardized using scikit-learn's `StandardScaler` ¹, which scales the data so that it's mean is 0, except for the Multinomial Naive Bayes, which expects counts and Random Forest, where the feature scale doesn't make a difference. The classifier uses the following features:

- **Focusing on length:** Sentence, word, and character count, mean word length and mean sentence length.
- **Focusing on punctuation:** The ratios of dots, commas, exclamation/question marks, colons and hyphens to total words.
- **Focusing on character case:** Ratios of all uppercase and title case words to other words.
- **Focusing on readability:** ARI.
- **Focusing on lexical diversity:** TTR, count of unique words, and Hapax-Legomenon rate.
- **Focusing on POS tags:** The ratios of all POS tags to total words. POS tags are taken from the Slovak WordNet ². WordNet words are used with diacritics, but if no match is found then they are used without diacritics as fallback.
- **Other:**
 - Ratio of stop words to total words. The list of Slovak stop words is publicly available on GitHub ³. The reviews are stripped of diacritics before checking for stop words to avoid missing any.
 - Count of unknown words. An word is considered unknown if it is not found in the WordNet even without diacritics.

¹<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler>

²<https://korpus.juls.savba.sk/WordNet.html>

³<https://github.com/stopwords-iso/stopwords-sk>

The TF-IDF classifier

Another base learner is a classifier which tries to predict the authenticity of the reviews upon their TF-IDF score. This is split from the textual stats classifier since the TF-IDF vector will be much larger than the vector of all other features, which are scalar values. This is done as to not „drown out“ other features, although since the two classifiers use mostly the same models, they could be merged. It uses scikit-learn’s `TfidfVectorizer` with a `ngram_range` setting of (1,2), which means it uses both unigrams and bigrams as features. The feature extraction uses the same stop words as the statistical one, where in this case the stop words are not taken into consideration. The feature extraction produces a sparse matrix which the model takes as input.

The Transformers classifier

The final base learner is an encoder language model fine-tuned for review classification. The purpose of this component is to find semantic giveaways and patterns in the text which could indicate the text being artificially generated. Since this is a language model, it is trained on the raw review texts and the class labels, (which are internally tokenized using the model’s tokenizer, turned into embeddings and then contextualized) and takes the raw text as input during inference. This part of the ensemble uses transformers’ `AutoModelForSequenceClassification` API and the model’s default tokenizer. It allows for any encoder model from the Hugging Face hub to be used, however during evaluation 2 models were tested:

1. *SlovakBERT*: A encoder model trained on several web crawls mainly containing the Slovak language from 2021. ⁴.
2. *ModernBERT*: A modernized version of BERT from 2024 ⁵.

5.2 The classification framework

The classifier is implemented as a Python library. It includes options to use several models on different parts of the ensemble, choose different training procedures and turn on and off different parts of the ensemble. It utilizes the *scikit-learn* ⁶ library for most models, *PyTorch* ⁷ for the MLP model and the *Hugging Face Transformers* ⁸ library for the Encoder model. The library has its core modules written as object-oriented and allows for easily adding another model, its configuration, or feature type.

5.3 Ensemble training algorithms

The library supports several algorithms for training the ensemble. The first, and the most simple one trains the base models on the training set, runs inference on the same data and then trains the meta model on those predictions. However, this can cause the base models

⁴<https://huggingface.co/gerulata/slovakbert>

⁵<https://huggingface.co/blog/modernbert>

⁶<https://scikit-learn.org/stable/>

⁷<https://pytorch.org/>

⁸<https://huggingface.co/docs/transformers/index>

to be overly accurate and overfit. The second algorithm uses a validation set, where it trains the base models on the training set, runs inference on the validation set and then trains the meta model on the inferred predictions. However, this has the disadvantage that it only uses the validation set predictions to train the meta model, which is only 10% of the entire dataset. The algorithm used for training the ensemble when evaluating is based on [9]. It uses *K-Fold cross validation* with $K = 5$ to train the base models. For each of the K training runs, it trains the base models on $K - 1$ folds. Then it runs inference of the base models on the remaining fold. The resulting predictions are used as $\frac{1}{K}$ of the resulting meta training set. This is done until inference is ran on all 5 folds, and the model predictions compose the training dataset. The last step is retraining the base models on the entire training set. This causes both the base and meta models to use 100% of the training data. Figure 10 showcases this on a high level:

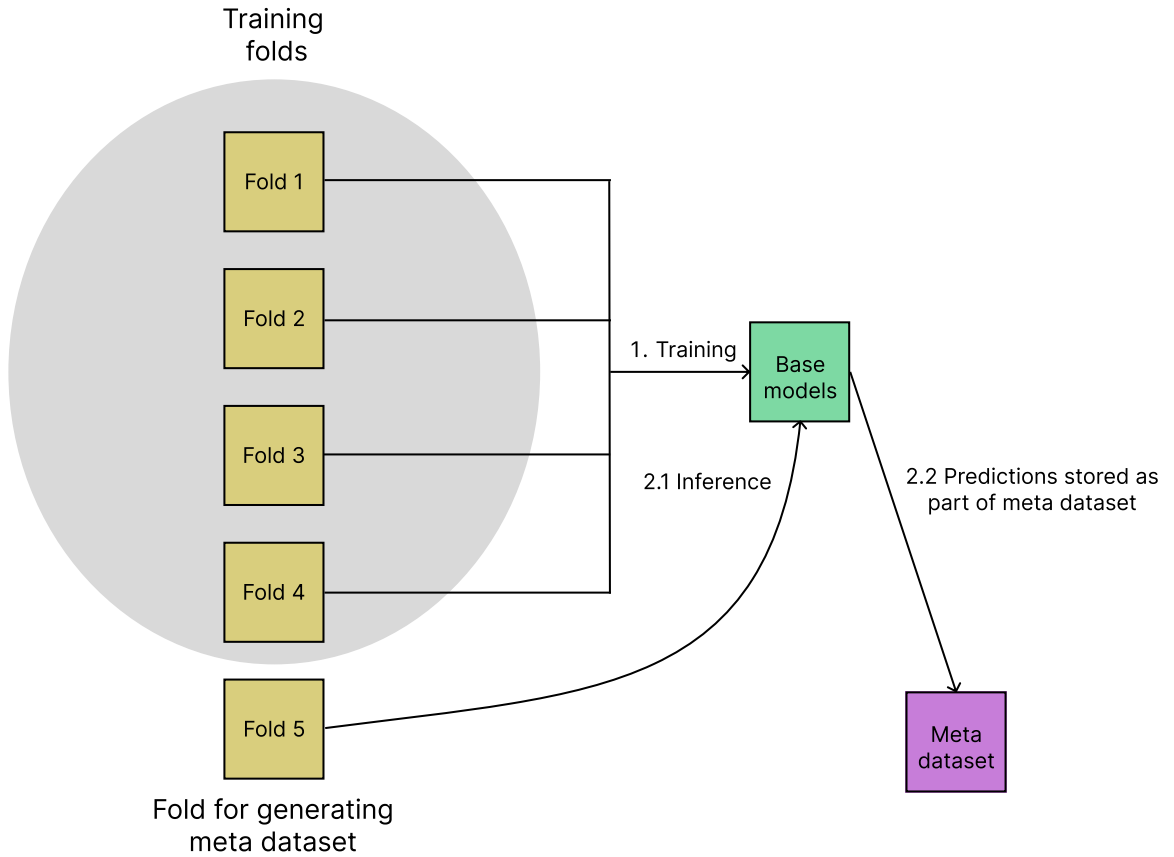


Figure 10: Base models are trained on $K - 1$ folds, and generate predictions on the remaining fold. This is repeated K times until the model generates predictions on all folds.

5.4 Used models

The library includes several available models, where each of them is configurable as per its most usual parameters. Each model also includes the part of the ensemble that the model can be used for. ?? shows which models are available in the library and in which part of the ensemble they can be used:

Model	Statistical	TF-IDF	Transformer	Meta
SVM	Yes	Yes	No	Yes
KNN	Yes	No	No	Yes
Multinomial Naive Bayes	No	Yes	No	No
Logistic Regression	Yes	Yes	No	Yes
Random Forest	Yes	No	No	Yes
PyTorch MLP	Yes	No	No	Yes
Hugging Face Transformer	No	No	Yes	No

Table 8: Models supported by the library and their supported features.

All models are configured to output probabilities instead of just binary class labels, 0.5 is used as the threshold for a review being fake. To some models this comes naturally, such as Logistic Regression or an MLP, but in the case of an SVM, the scikit-learn API does this by training an additional Logistic Regression model on the SVMs scores and fitting it with additional cross validation on the training data ⁹.

⁹<https://scikit-learn.org/stable/modules/svm>

Chapter 6

Evaluation

6.1 Individual model evaluation

Individual models were validated first to find the optimal components of the ensemble and find the optimal parameters for each model for each of its supported features.

Finding optimal model parameters

To find the best parameters for each model, the following was done:

1. Define parameter values to try out.
2. Create all possible combinations of those parameters for a given model.
3. Evaluate the model with the given parameters.
4. Choose the best performing combination.

This was done separately for both variants of the dataset. It was also done separately for all features the model supported, which means the optimal parameters could be different based on how the model was run, and with what data. For scikit-learn models, this was done using the `GridSearchCV` API ¹, which internally performs K-Fold Cross Validation. For Transformers models, it was done using Hugging Face's `hyperparameter_search` ² and for the PyTorch MLP it was done using a custom training loop which performed 5-fold cross-validation to evaluate. All models were evaluated using the F1-score, where for the MLP and scikit-learn models it was the average of the F1-scores across the K-runs, while for the transformer model it was the F1-score on the validation set.

Performance of individual models

The individual models were then evaluated. This was done as follows:

1. Re-train the models on the full training set using the found optimal parameters
2. Evaluate them on the testing set

The models were evaluated using the F1-Score, Recall, and Precision metrics. The results for each (*feature, variant*) pair are visible in [Figure 11](#). The first thing observed from the

¹https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV

²https://huggingface.co/docs/transformers/hpo_train

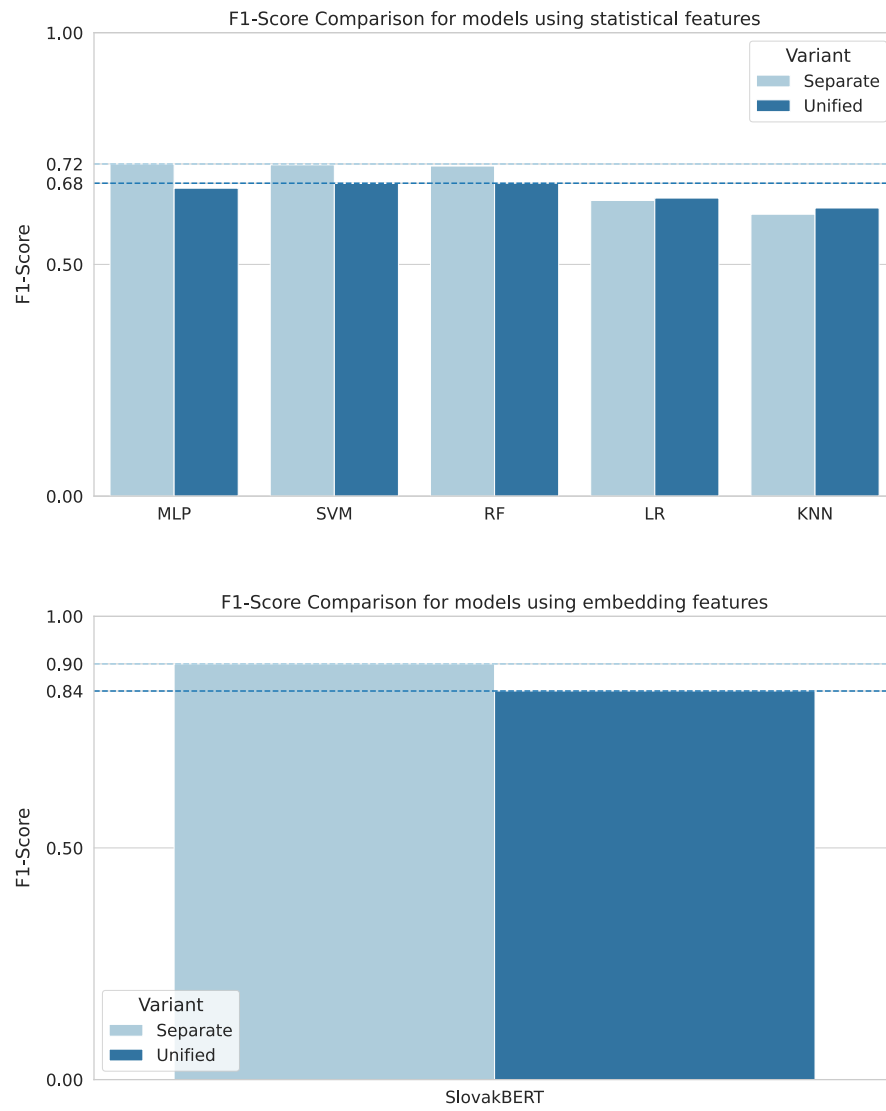


Figure 11: Comparison of F1-Scores across different features and dataset variants.

results was that the models reach higher scores in all but one case (Logistic Regression with statistical features). The obvious cause may be more training data, since it treats most reviews as 2 samples, but it could also be due to the separate review parts feeling more natural when they aren't concatenated together. Another thing to notice is that the SlovakBERT model outperforms all smaller models by a significant margin. This is no surprise, since the model is by far the most complex one, and „understands Slovak“ already, however it also comes at a significant computational cost. The models trained on TF-IDF come in second, with SVM being the top performer in both variants. The statistical models perform the worst, which may be due to the characteristic of the synthetic reviews: since the model that generated them was trained on the original reviews, it could cause them to match the originals in structure. Here, the PyTorch MLP performs best on the separate review parts, and Random Forest on the unified reviews. [Figure 12](#) shows another side-by-side comparison, comparing the Precision and Recall metrics of the models. Most models also visibly exhibit a higher Recall than Precision. This is most extreme in one case of the MLP model, where in one variant it almost reached a Recall of 1 while its precision was close to 0.5, which indicates just guessing fake each time. This is again more prevalent in the unified variant, probably due to less training samples. This could be a problem in production review systems, since it would flag many authentic reviews as fake, and keeping authentic reviews could be seen as a higher priority than removing fake ones.

Ensemble model choice

The following models were chosen for each of the features:

1. Statistical: Random Forest. It is very close to the best performing for this feature set. Compared to the other top performers (SVM and MLP), it is also more interpretable, computationally simpler, and has a more balanced Precision/Recall ratio across both variants.
2. TF-IDF: Logistic Regression. All models are very close in this variant concerning the F1-Score only, however LR has a slightly more balanced Precision/Recall ratio and is highly interpretable.
3. Transformer: SlovakBERT. Is smaller than ModernBERT and performs significantly better.

Feature importances of individual models

The features of individual models were evaluated for their importance where it was possible, so in the statistical and TF-IDF models. [Tables Table ??](#) and [Table ??](#) show the top 10 features for each dataset variant. The features are the coefficients of the Logistic Regression model trained on them, taken by the 10 largest absolute values. The features were extracted by fitting a `TfidfVectorizer` on the training set with the same parameters as when training, extracting each feature's name, mapping the coefficients of the LR model to them and getting the 10 most important ones.

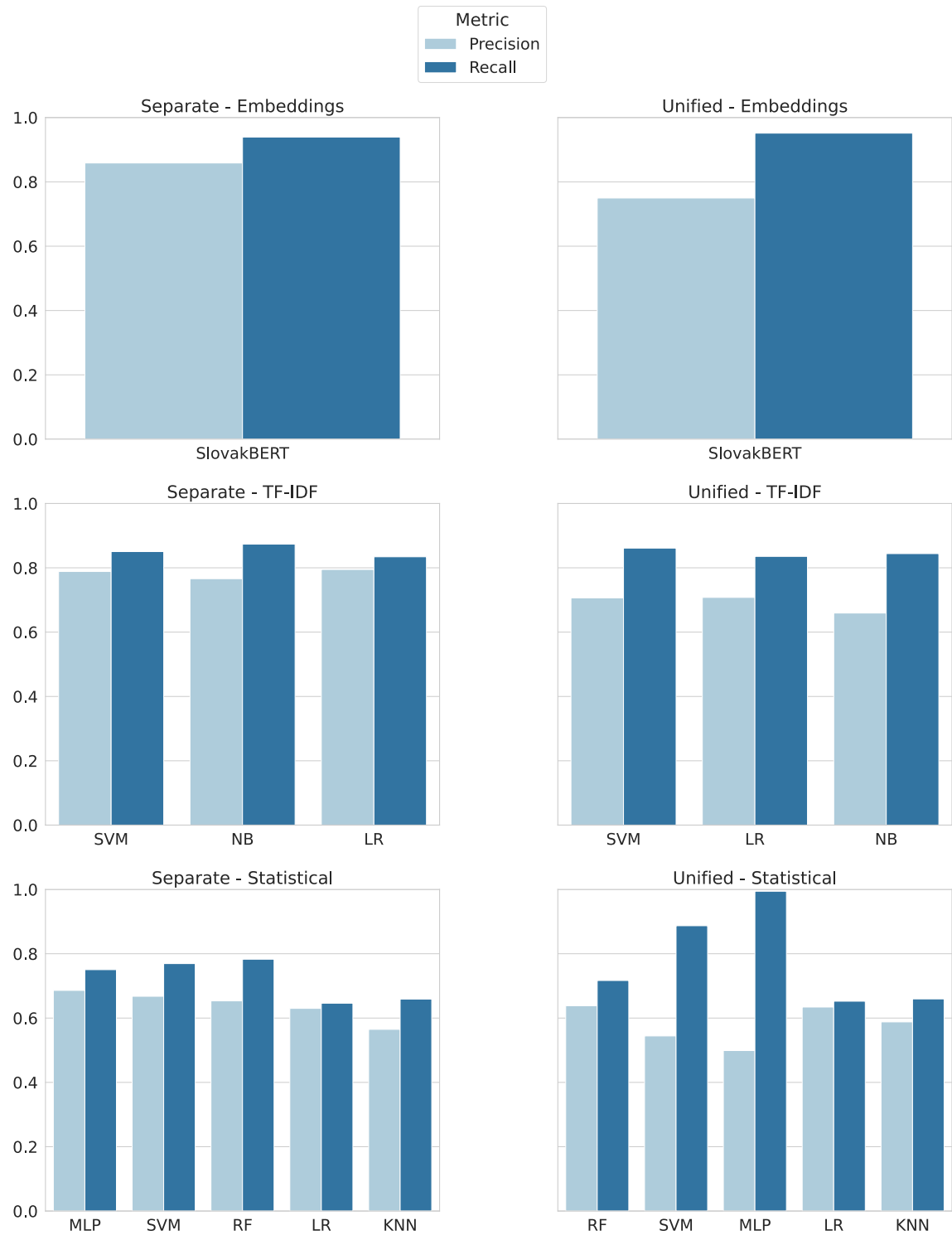


Figure 12: Precision and Recall of each model compared on each variant.

Feature	LR Coefficient
vynikajúci	7.070608
nocí	6.983378
hotely	6.830124
ďakujeme	6.768261
určite	6.434284
ktorý	6.142472
pomôct	6.110597
naozaj	6.073597
izba	5.944850
dispozícií	5.863855

Table 9: Top 10 most important TF-IDF features for the unified variant.

Feature	LR Coefficient
izbách	9.972730
hoteli	9.356958
priateľský	7.908222
sprcha	7.858609
sprche	7.694582
výhľad	7.337179
museli	7.139925
napriek	-7.313837
welnes	-7.726525
wellnes	-10.373896

Table 10: Top 10 most important TF-IDF features for the separate variant.

Interestingly enough, there is no overlap between the two variants. Also, the unified variant coefficients are all positive, meaning that they „nudge“ the model towards predicting the review as fake. All the words sound like something that would be expected in a hotel review. The statistical model’s feature importances were extracted using scikit-learn’s built-in `feature_importances_` API for the Random Forest model ³. **Figure 13** displays the 10 most important features for each dataset variant. As opposed to TF-IDF, the features have significant overlap among themselves. Features using punctuation are highly ranked, such as dot and comma ratios. Lexical diversity is also a strong indicator, with TTR and Hapax-Legomena rate both scoring highly. Features using raw counts such as sentence count and char count also score surprisingly highly, despite the model being prompted to match the original reviews in length.

Meta dataset

To avoid running the full pipeline including the base models for each full ensemble experiment, a *meta dataset* was generated from the predictions of the base models, as per the training algorithm. Then, a meta testing set was generated too, by combining the predic-

³https://scikit-learn.org/stable/auto_examples/ensemble/plot_forest_importances.html

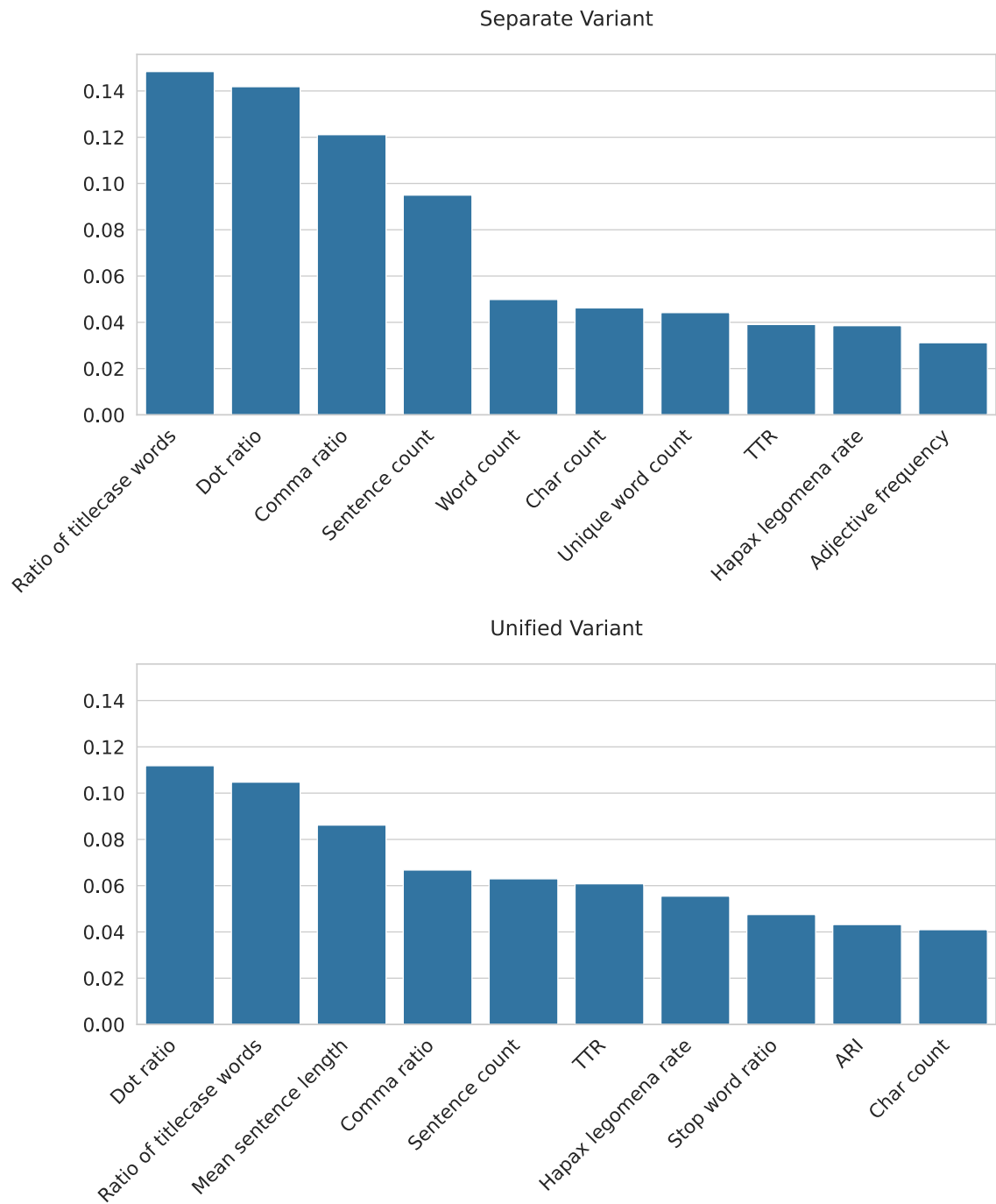


Figure 13: Top 10 features of the statistical model sorted by importance, for both variants.

tions of the trained models on the original testing set. This is equivalent to pausing the full ensemble pipeline at the beginning of the meta model training to store the dataset for faster subsequent training.

6.2 Full ensemble evaluation

As for the base learners, the optimal parameters were evaluated for the meta learner, following the same procedures. This was done on the full version of the meta dataset, containing predictions from all 3 base learners (twice for both variants of the original dataset). Then, the meta learners were evaluated on the meta testing set. The results are visible in [Table ??](#).

		Precision	Recall	F1-Score
Variant	Model			
Separate	SVM	0.891	0.907	0.899
	RF	0.896	0.900	0.898
	MLP	0.887	0.905	0.896
	LR	0.892	0.897	0.894
	KNN	0.892	0.890	0.891
Unified	RF	0.833	0.878	0.855
	MLP	0.807	0.901	0.851
	LR	0.822	0.868	0.845
	SVM	0.770	0.931	0.843
	KNN	0.816	0.838	0.827

Table 11: F1-Scores of the full ensemble.

They show that the entire ensemble doesn’t show a significant improvement over the individual peak performer (SlovakBERT) concerning F1-Score only. However, the results are much more balanced concerning the Precision/Recall ratio, as the two are almost the same for the top performing model across both variants, which is Random Forest. Due to this, the model was chosen for the ensemble’s meta model in other experiments.

6.3 Partial ensemble evaluation

The ensemble was also evaluated with each of it’s models „turned off“. The purpose of this was to find which models contribute most to the classifier, and which are redundant. [Figure 14](#) compares the F1-Scores, Precisions, and Recalls of the ensemble when all 3 models are used against when various models are turned off. The transformer model is visibly the most important model to the overall performance, since without it is basically the only time the ensemble sees a significant drop in performance. However, the Precision/Recall ratio can get more unstable without other models. This corresponds with [Figure 15](#), which shows the feature importance of the meta learner:

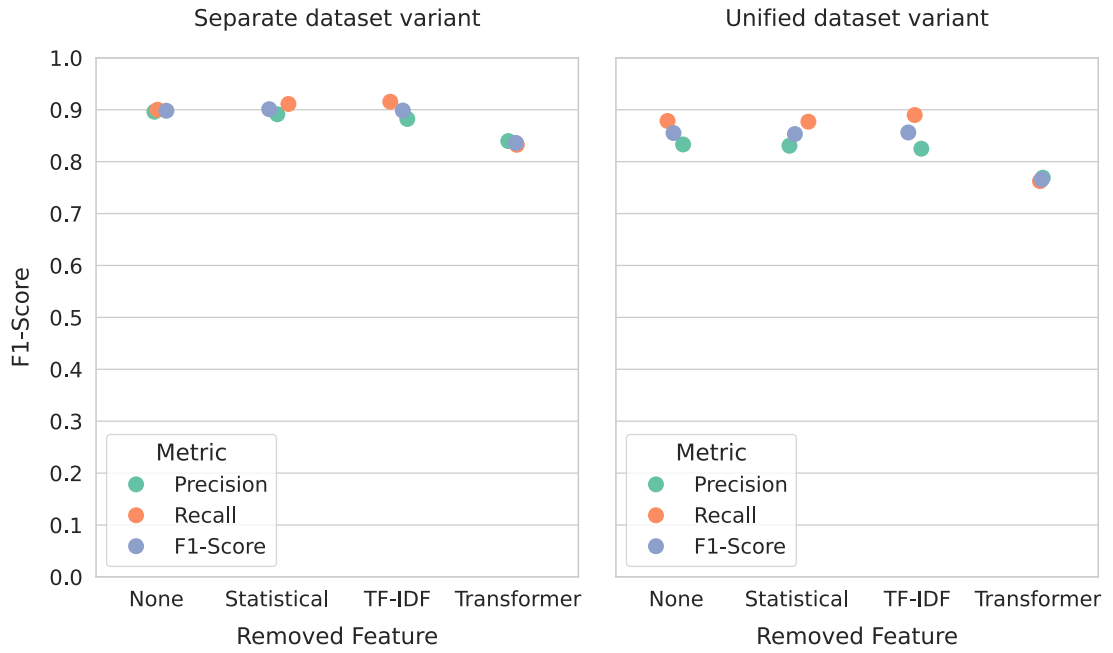


Figure 14: Precision, Recall, and F1-Score of the full and partial ensemble models.

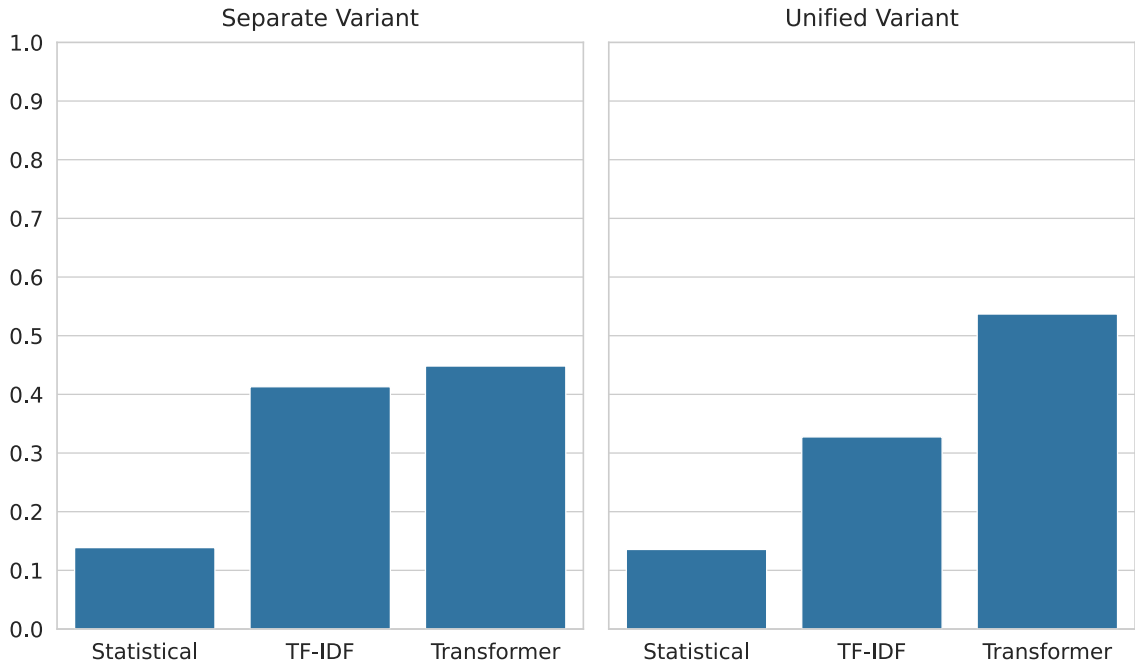


Figure 15: Feature importance of the meta model (Random Forest).

The Transformer model is the most important one in both variants, with the TF-IDF model coming close for the separate variant. Overall, the ensemble shows light improvements over individual models in F1-Score, but with a more balanced Precision/Recall ratio.

6.4 Other experiments

Other experiments were performed to test the model’s ability to generalize. These experiments always used the original training set of human written and Gemma 3 generated reviews. The testing set was modified via leaving the human written reviews unchanged, and either swapping or modifying the synthetic reviews to see how well would the model detect them. This was always done for both variants of the dataset. In addition to the Precision, Recall, and F1-Score metrics, this time the models were also evaluated for Accuracy.

SlovakMistral experiment

This experiment uses the `mistral-sk-7b` model, which was mentioned in [Section 4.4](#) as one of the models considered for generating the synthetic half of the dataset, where it has shown that it can produce quality Slovak, even scoring first on the taken Google Forms survey. For this experiment, new synthetic reviews were generated using the Mistral model. The count of the new reviews was approximately the count of synthetic reviews in the original testing set. Then, the original synthetic reviews from the testing set were swapped with the new Mistral fakes. The results can be visible in [Figure 16](#):

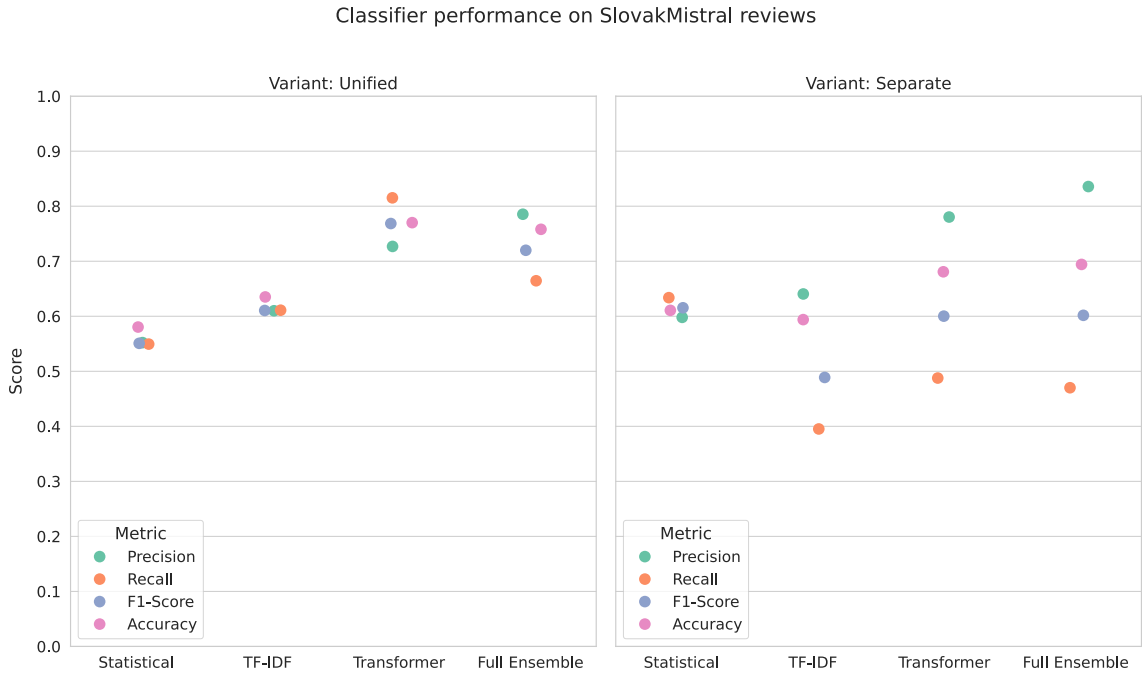


Figure 16: Comparison of F1-Score, Precision, and Recall on SlovakMistral reviews for the full ensemble and for the individual models.

We can see that the performance drops significantly from the Gemma 3 testing set. The drop off is (mostly) worse for the separate variant of the dataset than the unified one, perhaps due to lacking the connections between the positive and negative parts of the reviews. For the separate part, the F1-Score drops to approximately 0.6, which is mostly due to a steep drop-off in Recall, since the model still maintains a very high Precision (about 0.87), even higher than in the other variant which still maintains a decent performance. The Accuracy is also decent, but that might be due to correctly classifying the authentic reviews,

since the model only „catches“ about half of the SlovakMistral fakes ($Recall \approx 0.5$). The unified variant still maintains a relatively decent F1-Score (≈ 0.72). Interestingly enough, the Transformer model alone outperforms the full Ensemble concerning the F1-Score, even though it scores lower on Precision, which is already a previously observed trend, but with a much higher recall, catching more than 80% of SlovakMistral fakes. The lower performance of the full Ensemble than one of it’s parts might be amplified by the partial degradation of the performances of the Statistical and TF-IDF models, which may introduce noise to the predictions, the TF-IDF one in particular since it has shown a relatively high feature importance. The TF-IDF degradation may be due to the model using different words than fine-tuned Gemma 3.

Dataset without punctuation

This experiment removes all punctuation marks from the synthetic reviews. The removed characters are (’, ’, ’!, ’?, ’, ’, ’:, ’-), so all punctuation characters used in as features in the statistical model. Each synthetic review in the testing set had each of these marks removed. The results are visible in [Figure 17](#):

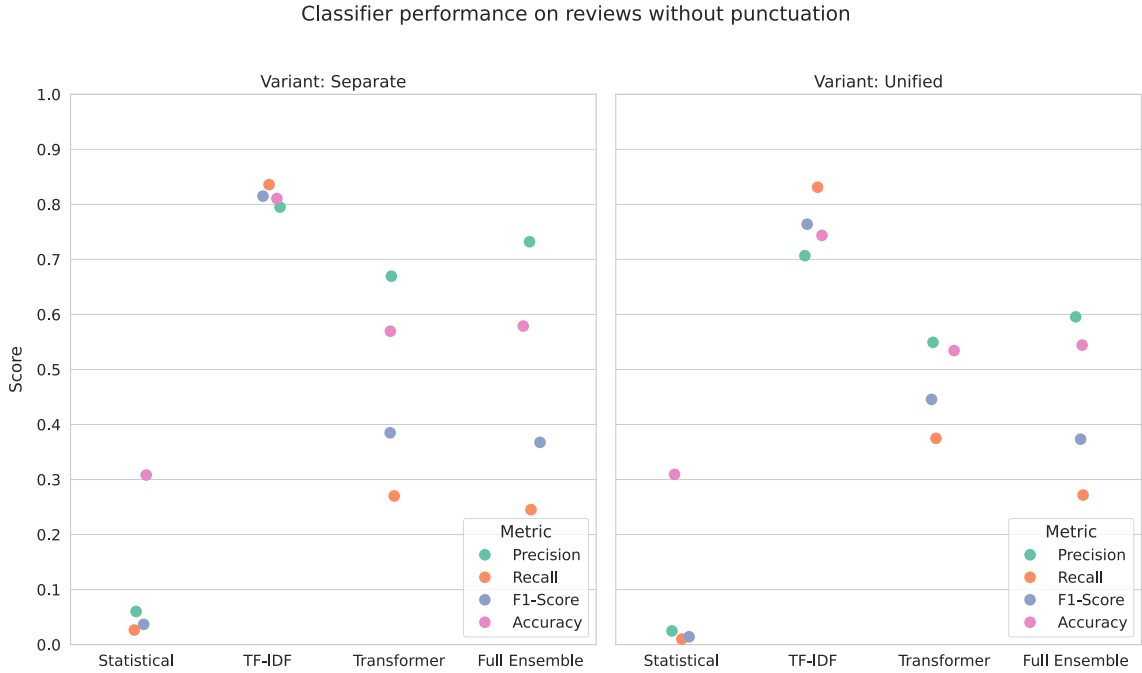


Figure 17: Comparison of F1-Score, Precision, and Recall on reviews without punctuation for the full ensemble and for the individual models.

Here we can see that the model surprisingly performs even worse than on the dataset which uses a completely different model, with a total collapse of the statistical model. The statistical drop off may be explained by the feature importance in [Figure 13](#), since dot and comma ratios were some of the most important features for the model, constantly setting them to zero might degrade it’s performance. While this is not entirely unexpected to some degree, what’s more surprising is the degradation of the transformer model, which only catches about $\frac{3}{10}$ to $\frac{4}{10}$ of fake reviews. As a result of that, the full Ensemble degrades with it, since the transformer is the most important base model during training as previously

seen in [Figure 15](#). However, the TF-IDF model still performs well, maintaining basically the same metrics for both variants as on the Gemma 3 testing set. This makes sense, since the TF-IDF implementation used in the classifier works with words and pairs of words. Some of them could have punctuation marks attached, (e.g. “hotel” and “hotel.”), but that will be only a small fraction.

Synonymized dataset

This experiment uses the Slovak WordNet, already mentioned in [Section 5.1](#). In addition to POS tags, it also contains synonyms for each word’s lemma. For this experiment, each synthetic review in the testing set had each of it’s word swapped with different probabilities if a match was found for it in the WordNet synsets. The results are visible in ??.

6.5 Practical applications

Chapter 7

Conclusion

Bibliography

- [1] Fake Review Statistics (2025): on Amazon Other Websites. *Capital One Shopping Research* online. 10. september 2025. Available at: <https://capitaloneshopping.com/research/fake-review-statistics>. [cit. 2025-10-05].
- [2] AHMED, A. et al. The Dead Internet Theory: Investigating the Rise of AI-Generated Content and Bot Dominance in Cyberspace. *Pakistan Journal of Engineering, Technology and Science* online. Karachi, Pakistan: Institute of Business Management, june 2024, vol. 12, no. 1, p. 37–48. ISSN 2224-2333. Available at: <https://doi.org/10.22555/pjets.v12i1.1077>. [cit. 2026-04-17].
- [3] AHMED, H. et al. Detection of Online Fake News Using N-Gram Analysis and Machine Learning Techniques. In: *Lecture Notes in Computer Science* online. Cham: Springer International Publishing, October 2017, p. 127–138. ISBN 9783319691541. Available at: https://doi.org/10.1007/978-3-319-69155-8_9. [cit. 2026-02-09].
- [4] ALAMAR, J. The Illustrated Transformer. *Jallamar.github.io* online. 27. june 2018. Available at: <https://jallamar.github.io/illustrated-transformer>. [cit. 2026-01-15].
- [5] ALAMAR, J. The Illustrated Word2vec. *Jallamar.github.io* online. 27. march 2019. Available at: <https://jallamar.github.io/illustrated-word2vec>. [cit. 2026-04-15].
- [6] AYYADEVARA, V. K. Basics of Machine Learning. In: *Pro Machine Learning Algorithms : A Hands-On Approach to Implementing Algorithms in Python and R* online. Berkeley, CA: Apress, 2018, p. 1–15. ISBN 978-1-4842-3564-5. Available at: https://doi.org/10.1007/978-1-4842-3564-5_1. [cit. 2026-04-26].
- [7] BAHETI, P. Train Test Validation Split: How To Best Practices. *V7labs.com* online. 13. september 2024. Available at: <https://www.v7labs.com/blog/train-validation-test-set>. [cit. 2026-01-18].
- [8] BROWNLEE, J. A Gentle Introduction to Bayes Theorem for Machine Learning. *Machinelearningmastery* online. 4. december 2019. Available at: <https://machinelearningmastery.com/bayes-theorem-for-machine-learning/>. [cit. 2026-04-04].
- [9] BROWNLEE, J. Stacking Ensemble Machine Learning With Python. *Machinelearningmastery* online. 27. april 2021. Available at: <https://machinelearningmastery.com/stacking-ensemble-machine-learning-with-python/>. [cit. 2026-04-13].

- [10] CUNNINGHAM, P. and DELANY, S. K-Nearest neighbour classifiers. *ACM Computing Surveys*. New York, United States: [b.n.], july 2021, vol. 54, no. 6, p. 1–25. ISSN 0360-0300.
- [11] DAS, M. et al. A Comparative Study on TF-IDF feature Weighting Method and its Analysis using Unstructured Dataset. online, august 2023. Available at: <https://doi.org/10.48550/arXiv.2308.04037>. [cit. 2026-02-09].
- [12] DEVLIN, J. et al. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)* online. Minneapolis, Minnesota: Association for Computational Linguistics, June 2019, p. 4171–4186. Available at: <https://doi.org/10.18653/v1/N19-1423>. [cit. 2025-10-05].
- [13] FANGFANG, L. What is logistic regression? *IBM* online. Available at: <https://www.ibm.com/think/topics/logistic-regression>. [cit. 2026-04-06].
- [14] FATIMA, N. Outliers detection and removal using IQR Method. *Medium.com* online. Available at: <https://medium.com/@noorfatimaafzalbutt/outliers-detection-and-removal-using-iqr-method-e629aa8089a8>. [cit. 2026-04-13].
- [15] GAMBETTI, A. and HAN, Q. *Combat AI With AI: Counteract Machine-Generated Fake Restaurant Reviews on Social Media* online. February 2023. Available at: <https://doi.org/10.48550/ARXIV.2302.07731>. [cit. 2025-09-21].
- [16] GIULIO, G. et al. Review bombing: ideology-driven polarisation in online ratings: The case study of The Last of Us (part II). *Quality Quantity* online. Heidelberg, Germany: Springer Science and Business Media, september 2024, vol. 59, no. 51, p. 315–341. ISSN 0033-5177. Available at: <https://doi.org/10.1007/s11135-024-01981-z>. [cit. 2026-04-13].
- [17] GUNDERSEN, G. Implicit Lifting and the Kernel Trick. *Gregorygundersen.com* online. 10. december 2019. Available at: <https://gregorygundersen.com/blog/2019/12/10/kernel-trick/>. [cit. 2026-04-17].
- [18] GUPTA, R. et al. Recent state-of-the-art of fake review detection: a comprehensive review. *The Knowledge Engineering Review* online. Fayetteville, GA, USA: Maximum Academic Press, 2024, vol. 39, p. e8. ISSN 0269-8889. Available at: <https://doi.org/10.1017/S0269888924000067>. [cit. 2026-01-16].
- [19] HAJEK, P. et al. Fake review detection in e-Commerce platforms using aspect-based sentiment analysis. *Journal of Business Research* online. Amsterdam, Netherlands: Elsevier BV, 2023, vol. 167, p. 114143. ISSN 0148-2963. Available at: <https://doi.org/10.1016/j.jbusres.2023.114143>. [cit. 2026-02-09].
- [20] HE, S. et al. The Market for Fake Reviews. *Marketing Science* online, 2022, vol. 41, no. 5, p. 896–921. ISSN 0732-2399. Available at: <https://doi.org/10.1287/mksc.2022.1353>. [cit. 2025-09-27].
- [21] ISLAM, M. et al. An Overview of Neural Network. *American Journal of Neural Networks and Applications* online. New York, United States: Science Publishing

- Group, 2019, vol. 5, no. 1, p. 7–11. ISSN 2469-7400. Available at: <https://doi.org/10.11648/j.ajna.20190501.12>. [cit. 2026-04-17].
- [22] JAIN, A. Everything about Random Forest. *Medium.com* online. 14. september 2024. Available at: <https://medium.com/@abhishekjainindore24/everything-about-random-forest-90c106d63989>. [cit. 2026-04-17].
- [23] JUREK LOUGHREY et al. A survey of commonly used ensemble-based classification techniques. *The Knowledge Engineering Review* online. Fayetteville, United States: [b.n.], november 2013, vol. 29, no. 5, p. 551–581. ISSN 0269-8889. Available at: <https://doi.org/10.1017/S0269888913000155>. [cit. 2026-04-18].
- [24] LAMBERT, M. From 2019 to 2024: AI-Generated Google Reviews Increased by 279.2 *Originality.ai* online. 11. october 2025. Available at: <https://originality.ai/blog/ai-google-reviews-study>. [cit. 2026-01-17].
- [25] LE, L. H. and HA, Q.-A. Effects of negative reviews and managerial responses on consumer attitude and subsequent purchase behavior: An experimental design. *Computers in Human Behavior* online. Amsterdam, Netherlands: Elsevier BV, 2021, vol. 124, p. 106912. ISSN 0747-5632. Available at: <https://doi.org/10.1016/j.chb.2021.106912>. [cit. 2025-09-21].
- [26] LUCA, M. Reviews, Reputation, and Revenue: The Case of Yelp.Com. *SSRN Electronic Journal* online. Boston: Harvard Business School, january 2010. 2011-10-11. Available at: <https://doi.org/10.2139/ssrn.1928601>. [cit. 2025-09-25].
- [27] MAJUMDAR, P. Type-Token Ratio (TTR) - A Measure of Lexical Diversity. online. Geneva, Switzerland: Zenodo, june 2025. Available at: <https://doi.org/10.5281/zenodo.15770103>. [cit. 2026-02-08].
- [28] MARCIANO, J. Fake online reviews cost \$152 billion a year. Here’s how e-commerce sites can stop them. *Weforum.org* online. 10. august 2021. Available at: <https://www.weforum.org/stories/2021/08/fake-online-reviews-are-a-152-billion-problem-heres-how-to-silence-them/>. [cit. 2025-09-26].
- [29] MIENYE, I. and JERE, N. A Survey of Decision Trees: Concepts, Algorithms, and Applications. *IEEE Access* online. New York, United States: Institute of Electrical and Electronics Engineers (IEEE), january 2024, vol. 12, p. 86716–86727. ISSN 2169-3536. Available at: <https://doi.org/10.1109/ACCESS.2024.3416838>.
- [30] MIENYE, I. and SUN, Y. A Survey of Ensemble Learning: Concepts, Algorithms, Applications, and Prospects. *IEEE Access* online. New York. United States: Institute of Electrical and Electronics Engineers (IEEE), september 2022, vol. 10, p. 99129–99149. ISSN 2169-3536. Available at: <https://doi.org/10.1109/ACCESS.2022.3207287>. [cit. 2026-04-18].
- [31] MIKOLOV, T. et al. Efficient Estimation of Word Representations in Vector Space. *Proceedings of Workshop at ICLR* online, january 2013. Available at: https://www.researchgate.net/publication/234131319_Efficient_Estimation_of_Word_Representations_in_Vector_Space. [cit. 2026-01-15].

- [32] MING, L. and MASSIMO, P. Data Augmentation for Fake Reviews Detection. In: *Proceedings of the Conference Recent Advances in Natural Language Processing - Large Language Models for Natural Language Processings* online. Shoumen, Bulgaria: INCOMA Ltd., January 2023, p. 673–680. Available at: https://doi.org/10.26615/978-954-452-092-2_073. [cit. 2026-01-17].
- [33] MIR, A. et al. Online Fake Review Detection Using Supervised Machine Learning And BERT Model. online. arXiv, january 2023. Available at: <https://doi.org/10.48550/arXiv.2301.03225>. [cit. 2026-02-23].
- [34] MOHAWESH, R. et al. Fake Reviews Detection: A Survey. *IEEE Access* online. New York, United States: Institute of Electrical and Electronics Engineers (IEEE), 2021, vol. 9, p. 65771–65802. ISSN 2169-3536. Available at: <https://doi.org/10.1109/ACCESS.2021.3075573>. [cit. 2025-09-21].
- [35] MUKHERJEE, A. et al. What Yelp Fake Review Filter Might Be Doing? *Proceedings of the 7th International Conference on Weblogs and Social Media, ICWSM 2013* online, january 2013, vol. 7, no. 1, p. 409–418. ISSN 2334-0770. Available at: <https://doi.org/10.1609/icwsm.v7i1.14389>. [cit. 2026-01-17].
- [36] OPARA, C. StyloAI: Distinguishing AI-Generated Content with Stylometric Analysis. *ArXiv (Cornell University)* online, may 2024. Available at: <https://doi.org/10.48550/arXiv.2405.10129>. [cit. 2026-04-13].
- [37] PAGET, S. Local Consumer Review Survey 2025. *Brightlocal.com* online. 29. january 2025. Available at: <https://www.brightlocal.com/research/local-consumer-review-survey-2025/>. [cit. 2025-09-21].
- [38] PENG, J.; LEE, K. and INGERSOLL, G. An Introduction to Logistic Regression Analysis and Reporting. *Journal of Educational Research - J EDUC RES* online. London, United Kingdom: Informa UK Limited, september 2002, vol. 96, p. 3–14. Available at: <https://doi.org/10.1080/00220670209598786>. [cit. 2026-04-06].
- [39] RADFORD, A. and NARASIMHAN, K. *Improving Language Understanding by Generative Pre-Training* online. June 2018. Available at: <https://api.semanticscholar.org/CorpusID:49313245>. [cit. 2025-10-05].
- [40] RASCHKA, S. An Overview of General Performance Metrics of Binary Classifier Systems. online, october 2014. Available at: <https://doi.org/10.13140/2.1.4639.7440>. [cit. 2026-04-18].
- [41] RENNIE, J. D. M.; SHIH, L.; TEEVAN, J. and KARGER, D. R. Tackling the poor assumptions of naive bayes text classifiers. In: *Proceedings of the Twentieth International Conference on International Conference on Machine Learning* online. Washington, DC, USA: AAAI Press, 2003, p. 616–623. ISBN 1577351894. Available at: <https://dl.acm.org/doi/10.5555/3041838.3041916>. [cit. 2026-04-04].
- [42] SCHUFF, D. et al. Understanding the Review Bombing Phenomenon in Movies and Television. *Proceedings of the 57th Hawaii International Conference on System Sciences* online, january 2024. ISSN 1530-1605. Available at: <https://doi.org/10.24251/hicss.2024.036>. [cit. 2026-04-13].

- [43] SHMILOVICI, A. Support Vector Machines. In: MAIMON, O. and ROKACH, L., ed. *Data Mining and Knowledge Discovery Handbook* online. Boston, MA: Springer US, 2005, p. 257–276. ISBN 978-0-387-25465-4. Available at: https://doi.org/10.1007/0-387-25465-X_12. [cit. 2026-04-03].
- [44] THAKUR, A. and KONDE, A. Fundamentals of Neural Networks. *International Journal for Research in Applied Science and Engineering Technology* online. Sonipat, Haryana, India: International Journal for Research in Applied Science and Engineering Technology (IJRASET), august 2021, vol. 9, no. 8, p. 407–426. ISSN 2321-9653. Available at: <https://doi.org/10.22214/ijraset.2021.37362>. [cit. 2026-04-17].
- [45] THEUERKAUF, R. and PETERS, R. Detecting Fake Reviews: Just a Matter of Data. In: *Proceedings of the Annual Hawaii International Conference on System Sciences* online. Lahaina, Hawaii: Hawaii International Conference on System Sciences, January 2023. ISSN 2572-6862. Available at: <https://doi.org/10.24251/HICSS.2023.534>. [cit. 2026-02-23].
- [46] USMAN, U. Identifying Fake Reviews on Amazon Through Sentiment and Linguistic Cues. online, august 2025. Available at: <https://www.researchgate.net/publication/394533599>. [cit. 2026-02-09].
- [47] VASWANI, A. et al. Attention is All you Need. In: GUYON, I.; LUXBURG, U. V.; BENGIO, S.; WALLACH, H.; FERGUS, R. et al., ed. *Advances in Neural Information Processing Systems* online. Red Hook, New York: Curran Associates, Inc., 2017, vol. 30, p. 5998 – 6008, 2023-08-02. Available at: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf. [cit. 2025-09-28].
- [48] VIMALADEVI, K. and DHANABHAKAYM, M. A Study on the Effects of Online Consumer Reviews on Purchasing Decision. *Prestige International Journal of Management IT- Sanchayan* online, 2012, vol. 1, no. 1, p. 91–99. ISSN 2277-1689. Available at: <https://doi.org/10.37922/PIJMIT.2012.V01i01.005>. [cit. 2025-09-22].
- [49] ZHOU, Z.-H. Introduction. In: *Machine Learning* online. Singapore: Springer Singapore, 2021, p. 1–24. ISBN 978-981-15-1967-3. Available at: https://doi.org/10.1007/978-981-15-1967-3_1. [cit. 2026-04-26].
- [50] ÖZAYDIN, H. Fake Reviews and Ratings Undermining Consumer Trust. In: ÖZKAYNAR, K. and ABBASOĞLU, , ed. *Consumer, Marketing, AI: Dark Sides and Ethics* online. Gaziantep, Turkey: Özgür Yayınları, March 2025. ISBN 9786255958723. Available at: <https://doi.org/10.58830/ozgur.pub710.c3028>. [cit. 2025-09-21].

Appendix A

Hyperparameters of used models

This appendix describes the parameters of the models used for synthesis and the parameters of the classification models, resulted from cross validation.

Hyperparameters of Gemma 3 for review synthesis

The process of creating synthetic reviews was done with the Gemma 3 model. The process used the following hyperparameters:

```
{
  "lora": {
    "r": 8,
    "alpha": 16,
    "dropout": 0,
    "bias": "none",
    "target_modules": ["q_proj", "k_proj", "v_proj",
                      "o_proj", "gate_proj", "up_proj", "down_proj"]
  },
  "training": {
    "epochs": 1,
    "batch_size": 2,
    "gradient_accumulation_steps": 8,
    "learning_rate": 5e-5,
    "optimizer": "adamw_8bit",
    "weight_decay": 0.01,
    "lr_scheduler": "linear",
    "warmup_steps": 100,
    "seed": 42
  }
}
```


And for generating synthetic reviews, the following ones were used:

```
{
  "do_sample": True,
  "repetition_penalty": uniform(1.2, 1.5),
  "temperature": 1.0,
  "top_p": 0.95,
  "top_k": 64
}
```

Hyperparameters of used classifiers

This section describes the hyperparameters of all classifiers, for both dataset variants and for all of their supported features. Each entry is a classifier, with entries for both dataset variants („separate” and „unified”). Then each of these entries has a entry for all features that the classifier supports („statistical”, „tfidf”, „transformer”, and „meta”).

```
{
  "knn": {
    "separate": {
      "statistical": {
        "algorithm": "auto",
        "metric": "manhattan",
        "n_neighbors": 7,
        "weights": "uniform"
      },
      "meta": {
        "algorithm": "auto",
        "metric": "euclidean",
        "n_neighbors": 7,
        "weights": "uniform"
      }
    },
    "unified": {
      "statistical": {
        "algorithm": "auto",
        "metric": "manhattan",
        "n_neighbors": 7,
        "weights": "uniform"
      },
      "meta": {
        "algorithm": "auto",
        "metric": "manhattan",
        "n_neighbors": 7,
        "weights": "uniform"
      }
    }
  },
  "logistic_regression": {
```

```

"separate": {
  "statistical": {
    "C": 10.0,
    "l1_ratio": 1.0,
    "solver": "liblinear"
  },
  "tfidf": {
    "C": 10.0,
    "l1_ratio": 0.0,
    "solver": "liblinear"
  },
  "meta": {
    "C": 1.0,
    "l1_ratio": 0.0,
    "solver": "liblinear"
  }
},
"unified": {
  "statistical": {
    "C": 10.0,
    "l1_ratio": 1.0,
    "solver": "liblinear"
  },
  "tfidf": {
    "C": 10.0,
    "l1_ratio": 0.0,
    "solver": "liblinear"
  },
  "meta": {
    "C": 0.1,
    "l1_ratio": 0.0,
    "solver": "liblinear"
  }
}
},
"mlp": {
  "separate": {
    "statistical": {
      "hidden_layer_sizes": [
        128,
        64,
        32
      ],
      "activation": "tanh",
      "optimizer": "adam",
      "learning_rate": 0.001,
      "epochs": 200
    },

```

```

        "meta": {
            "hidden_layer_sizes": [
                128,
                64,
                32
            ],
            "activation": "tanh",
            "optimizer": "adam",
            "learning_rate": 0.0003,
            "epochs": 100
        }
    },
    "unified": {
        "statistical": {
            "hidden_layer_sizes": [
                32,
                16
            ],
            "activation": "relu",
            "optimizer": "sgd",
            "learning_rate": 0.0001,
            "epochs": 50
        },
        "meta": {
            "hidden_layer_sizes": [
                64,
                32
            ],
            "activation": "tanh",
            "optimizer": "adam",
            "learning_rate": 0.0003,
            "epochs": 200
        }
    }
},
"modernbert": {
    "separate": {
        "transformer": {
            "learning_rate": 5e-05,
            "num_train_epochs": 4,
            "per_device_train_batch_size": 32,
            "per_device_eval_batch_size": 32
        }
    },
    "unified": {
        "transformer": {
            "learning_rate": 5e-05,
            "num_train_epochs": 4,

```

```

        "per_device_train_batch_size": 32,
        "per_device_eval_batch_size": 32
    }
}
},
"naive_bayes": {
    "separate": {
        "tfidf": {
            "alpha": 0.1,
            "fit_prior": false
        }
    },
    "unified": {
        "tfidf": {
            "alpha": 0.1,
            "fit_prior": true
        }
    }
},
"random_forest": {
    "separate": {
        "statistical": {
            "max_depth": 10,
            "max_features": "log2",
            "min_samples_leaf": 2,
            "min_samples_split": 10,
            "n_estimators": 200
        },
        "meta": {
            "max_depth": 10,
            "max_features": "sqrt",
            "min_samples_leaf": 2,
            "min_samples_split": 10,
            "n_estimators": 100
        }
    },
    "unified": {
        "statistical": {
            "max_depth": 10,
            "max_features": "log2",
            "min_samples_leaf": 4,
            "min_samples_split": 2,
            "n_estimators": 200
        },
        "meta": {
            "max_depth": 10,
            "max_features": "sqrt",
            "min_samples_leaf": 4,

```

```

        "min_samples_split": 10,
        "n_estimators": 200
    }
}
},
"slovakbert": {
    "separate": {
        "transformer": {
            "learning_rate": 5e-05,
            "num_train_epochs": 3,
            "per_device_train_batch_size": 32,
            "per_device_eval_batch_size": 16
        }
    },
    "unified": {
        "transformer": {
            "learning_rate": 3e-05,
            "num_train_epochs": 5,
            "per_device_train_batch_size": 32,
            "per_device_eval_batch_size": 16
        }
    }
},
"svm": {
    "separate": {
        "statistical": {
            "C": 1.0,
            "gamma": "scale",
            "kernel": "rbf"
        },
        "tfidf": {
            "C": 1.0,
            "gamma": "scale",
            "kernel": "sigmoid"
        },
        "meta": {
            "C": 1.0,
            "gamma": "scale",
            "kernel": "rbf"
        }
    },
    "unified": {
        "statistical": {
            "C": 1.0,
            "degree": 4,
            "kernel": "poly"
        },
        "tfidf": {

```

```

        "C": 1.0,
        "gamma": "scale",
        "kernel": "sigmoid"
    },
    "meta": {
        "C": 1.0,
        "gamma": "auto",
        "kernel": "rbf"
    }
}
}
}
}

```

Appendix B

Guideline to reproducing experiments

This chapter shows how to run the experiments used to evaluate the classification models. To successfully run the experiments, the following prerequisites need to be met:

1. You have `Python 3.14` on your system with a package manager (either `pip` or `uv`), the `review-classifier` repository cloned, and a set up virtual environment with the repository's dependencies (can be found in the repository root in `pyproject.toml` and installed with `uv sync`).
2. You have the synthetic review dataset splits locally, with both the unified and separate variants, and with folders for a 90-10 train/test split and a K-Fold split.
3. You have the variants of the testing set used in the

Meta dataset generation and evaluation on testing set

To reproduce these results, first generate the meta dataset. This can be done with:

Then, you can test the meta models on the testing set with:

An alternative approach is to use the full pipeline, and do this in one run, although it is not the way it was done when conducting the experiments. This can be done with:

The Slovak Mistral experiment

Experiment on synonymized testing set

Experiment on testing set without punctuation